



Embedded Signal Processing Systems

International Master of Science in Electrical Engineering

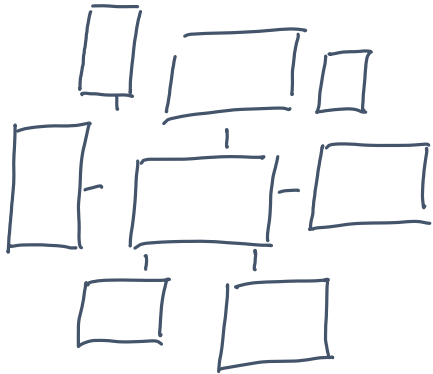
Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit



Introduction to DDFS Architectures

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Introduction to DDFS Architectures

Today's Agenda

- More on DDFS architecture
- Logic simulation using Mentor Modelsim
- Lab@Home Exercises



Introduction to DDFS Architectures

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Lecture Review - Direct Digital Frequency Synthesis

- Digital signal processing, as it is used in various industrial applications, often requires the implementation of special functions in addition to filtering.

Lecture Review - Direct Digital Frequency Synthesis

- Digital signal processing, as it is used in various industrial applications, often requires the implementation of special functions in addition to filtering.
- Examples are **modulation, demodulation, feature detection, and frequency translation.**

Lecture Review - Direct Digital Frequency Synthesis

- Digital signal processing, as it is used in various industrial applications, often requires the implementation of special functions in addition to filtering.
- Examples are modulation, demodulation, feature detection, and frequency translation.
- Often, there are several ways in which a particular function can be implemented: One method may require less computation or resources than another.

Lecture Review - Direct Digital Frequency Synthesis

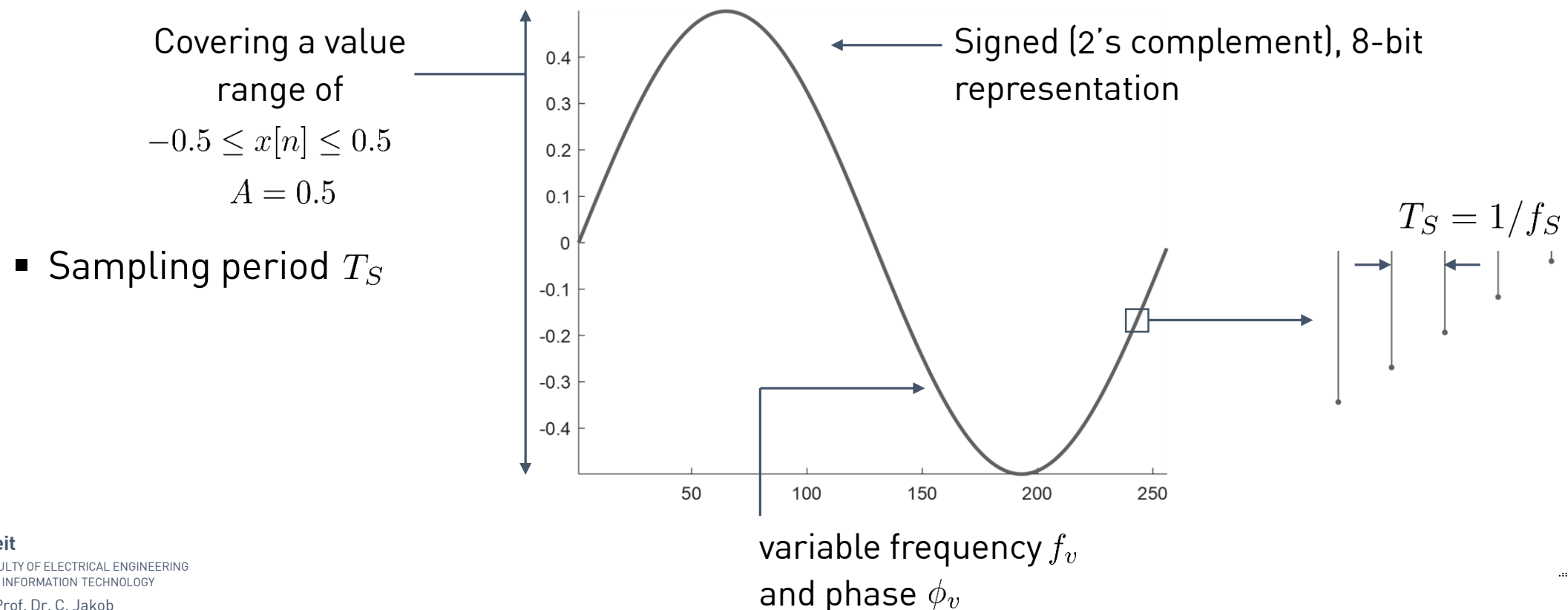
- Digital signal processing, as it is used in various industrial applications, often requires the implementation of special functions in addition to filtering.
- Examples are modulation, demodulation, feature detection, and frequency translation.
- Often, there are several ways in which a particular function can be implemented: One method may require less computation or resources than another.
- It is important to recognize the many methods to reduce the computational effort associated with a dedicated algorithmic implementation, because these approaches often make the difference between a practical system and one that is unrealistically complex ...

Lecture Review - Direct Digital Frequency Synthesis

- Digital signal processing, as it is used in various industrial applications, often requires the implementation of special functions in addition to filtering.
- Examples are modulation, demodulation, feature detection, and frequency translation.
- Often, there are several ways in which a particular function can be implemented: One method may require less computation or resources than another.
- It is important to recognize the many methods to reduce the computational effort associated with a dedicated algorithmic implementation, because these approaches often make the difference between a practical system and one that is unrealistically complex ...
- In the next lectures, we discuss various method for **synthesizing digital (sinusoidal) signals**

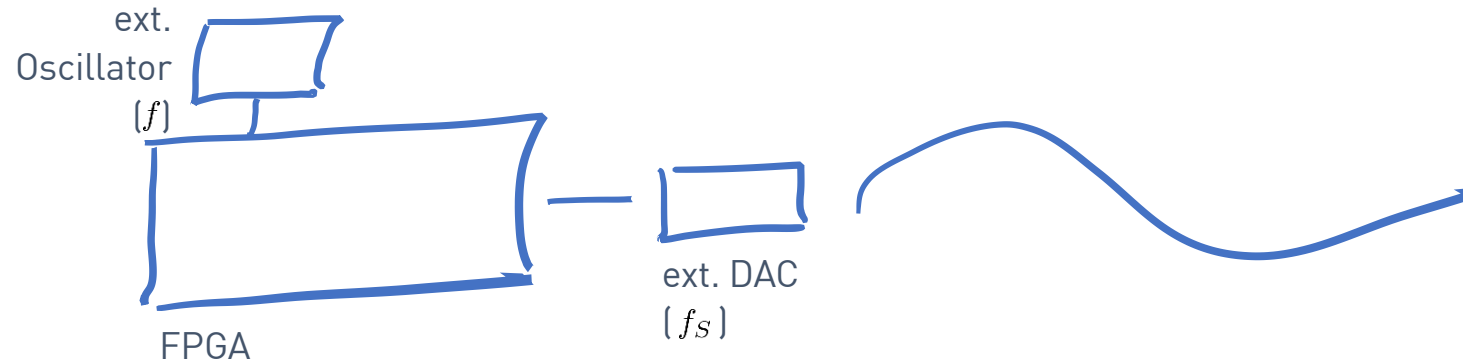
Lecture Review - Direct Digital Frequency Synthesis

- How to generate a digital sine wave signal?
- Assume $x[n] = A \sin(2\pi f_v n T_S + \phi_v)$



Lecture Review - Direct Digital Frequency Synthesis

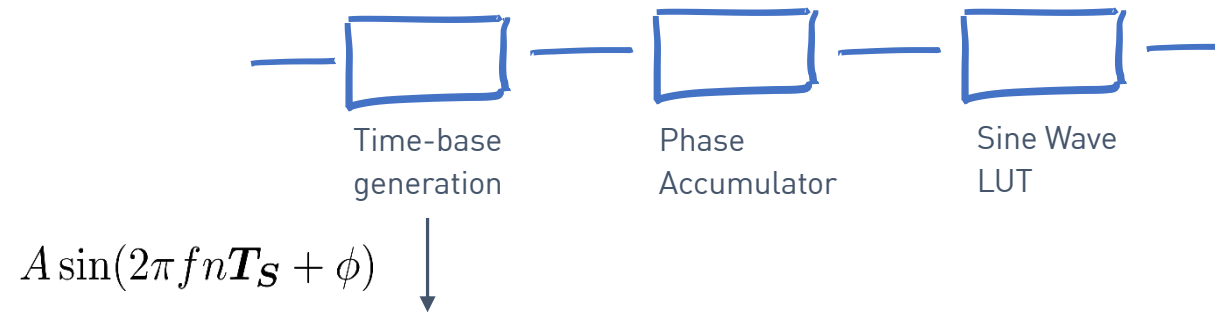
- Assume the following system structure ...



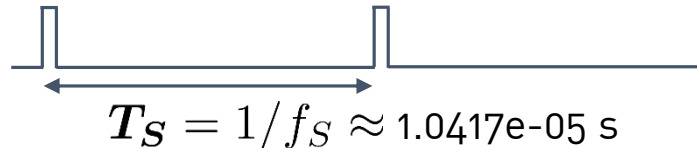
- External oscillator: f - 50MHz
- Sampling frequency: f_s - 96kHz

Lecture Review - Direct Digital Frequency Synthesis

- System internals – a closer perspective ...



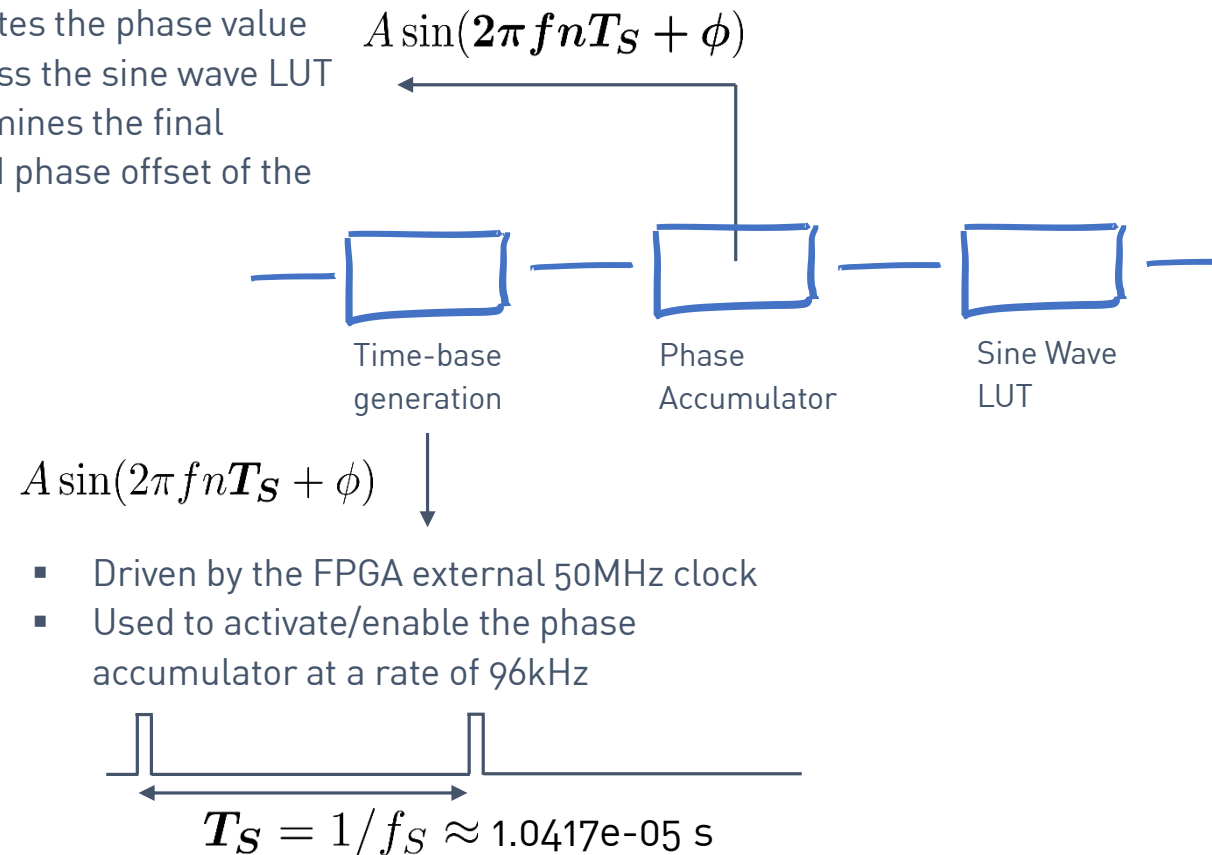
- Driven by the FPGA external 50MHz clock
- Used to activate/enable the phase accumulator at a rate of 96kHz



Lecture Review - Direct Digital Frequency Synthesis

- System internals – a closer perspective ...

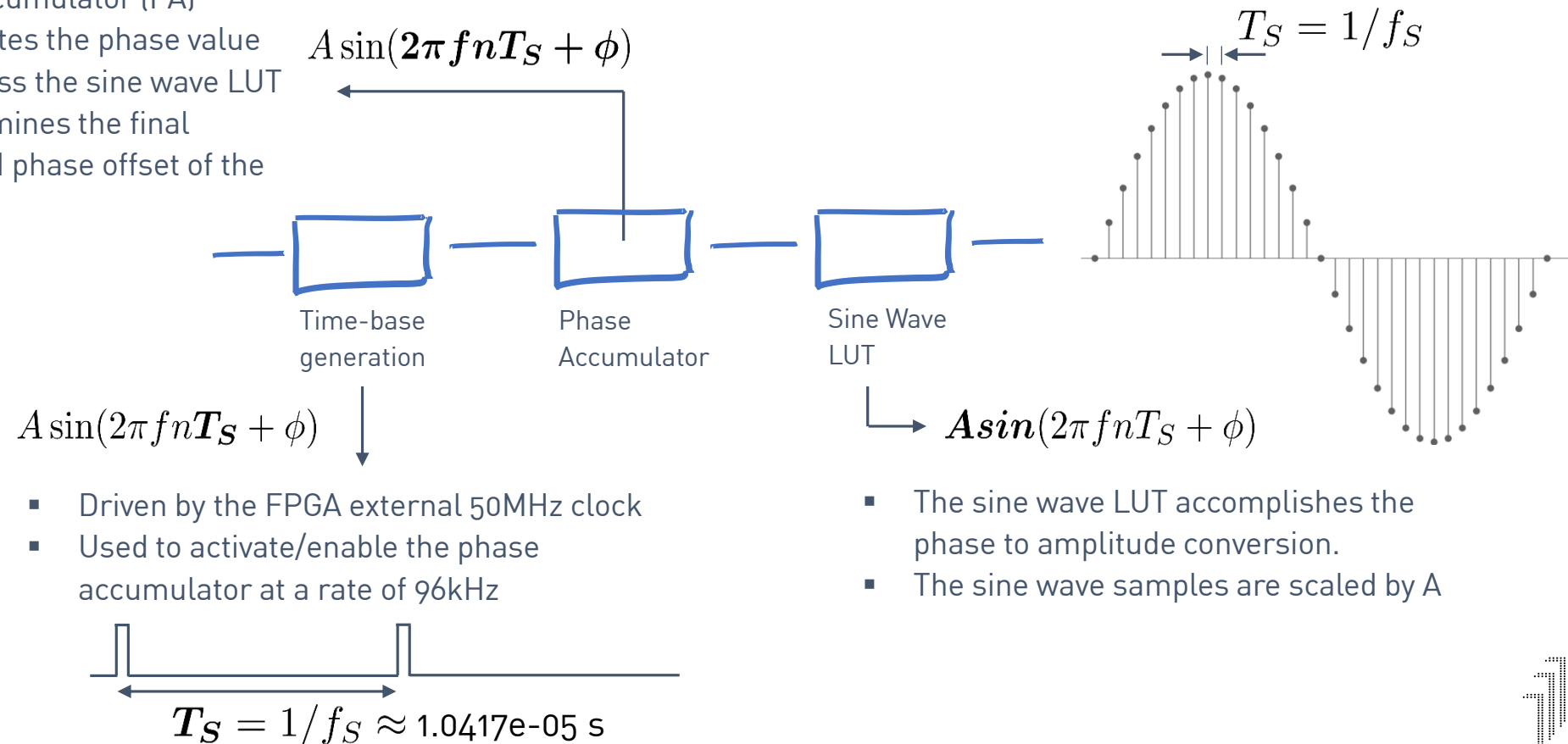
- The phase accumulator (PA) finally generates the phase value used to address the sine wave LUT
- The PA determines the final frequency and phase offset of the output signal



Lecture Review - Direct Digital Frequency Synthesis

■ System internals – a closer perspective ...

- The phase accumulator (PA) finally generates the phase value used to address the sine wave LUT
- The PA determines the final frequency and phase offset of the output signal



- Driven by the FPGA external 50MHz clock
- Used to activate/enable the phase accumulator at a rate of 96kHz

- The sine wave LUT accomplishes the phase to amplitude conversion.
- The sine wave samples are scaled by A

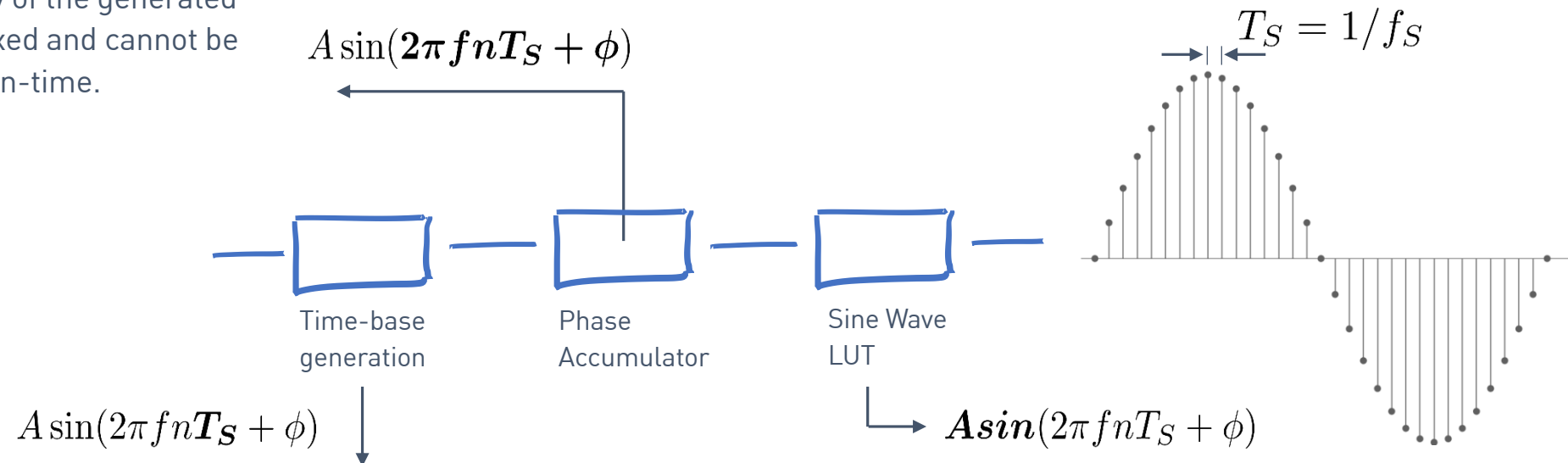
Lecture Review - Direct Digital Frequency Synthesis

- Let's summarize the main characteristics of the previously developed system:

Lecture Review - Direct Digital Frequency Synthesis

- Let's summarize the main characteristics of the previously developed system:

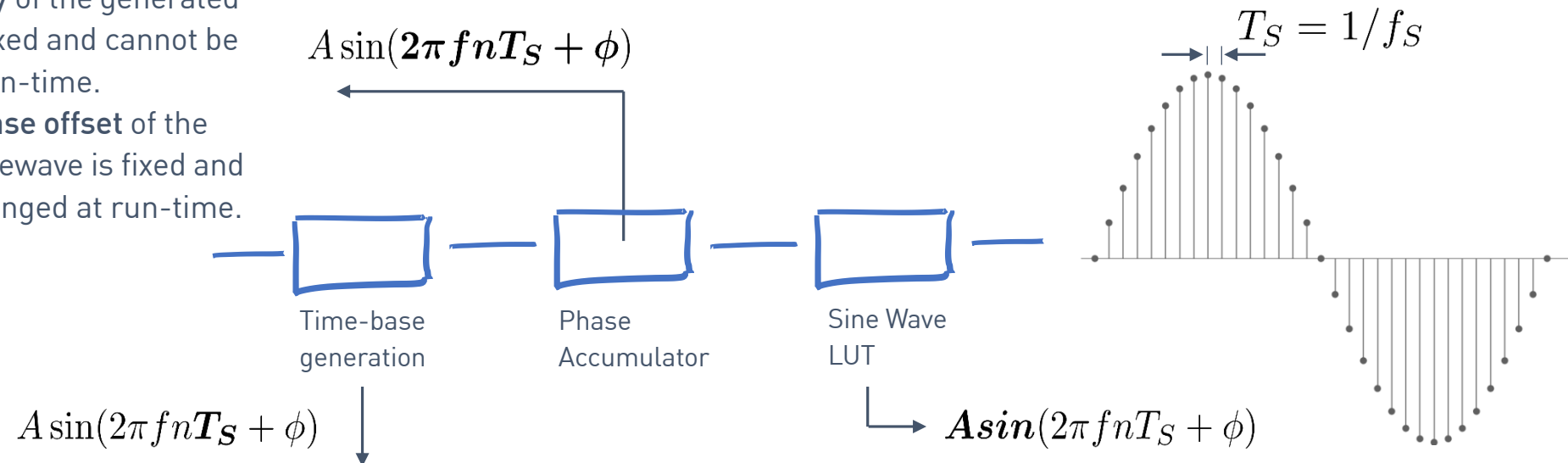
- The frequency of the generated sinewave is fixed and cannot be changed at run-time.



Lecture Review - Direct Digital Frequency Synthesis

- Let's summarize the main characteristics of the previously developed system:

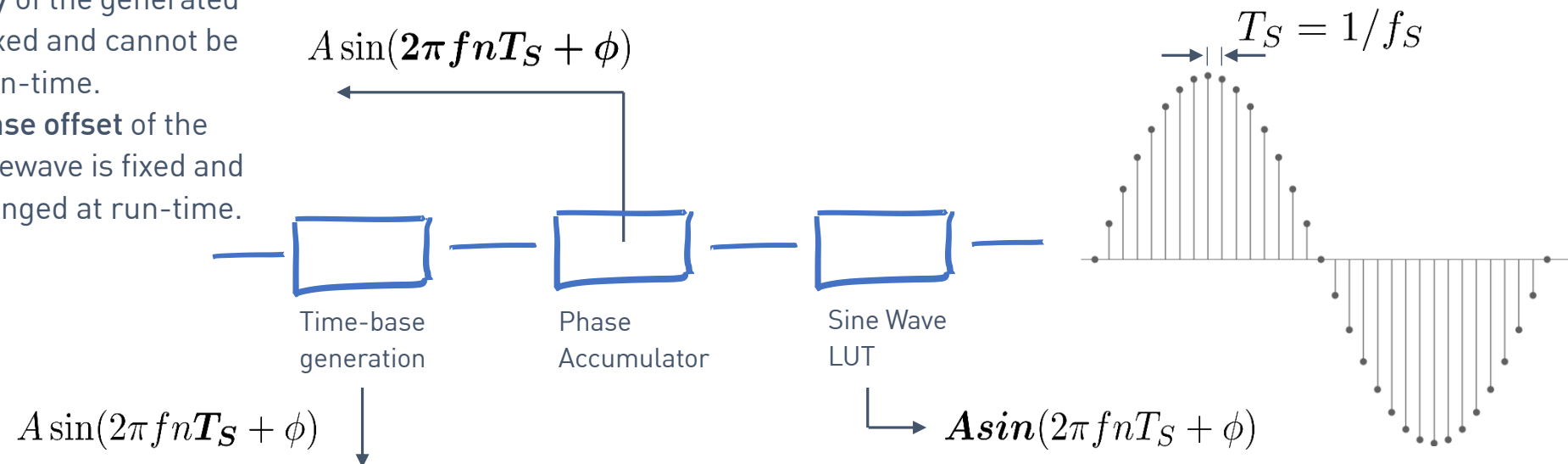
- 1 The **frequency** of the generated sinewave is fixed and cannot be changed at run-time.
- 2 The initial **phase offset** of the generated sinewave is fixed and cannot be changed at run-time.



Lecture Review - Direct Digital Frequency Synthesis

- Let's summarize the main characteristics of the previously developed system:

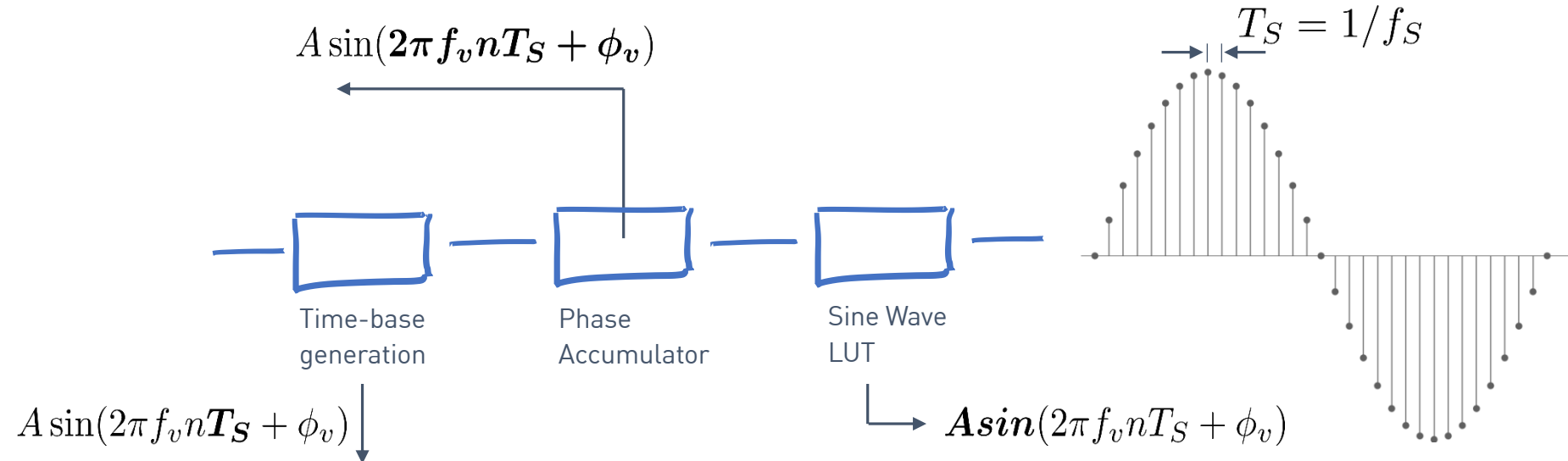
- 1 The **frequency** of the generated sinewave is fixed and cannot be changed at run-time.
- 2 The initial **phase offset** of the generated sinewave is fixed and cannot be changed at run-time.



- 3 A complete cycle sine wave is stored in read-only memory (ROM), or a lookup table (LUT),

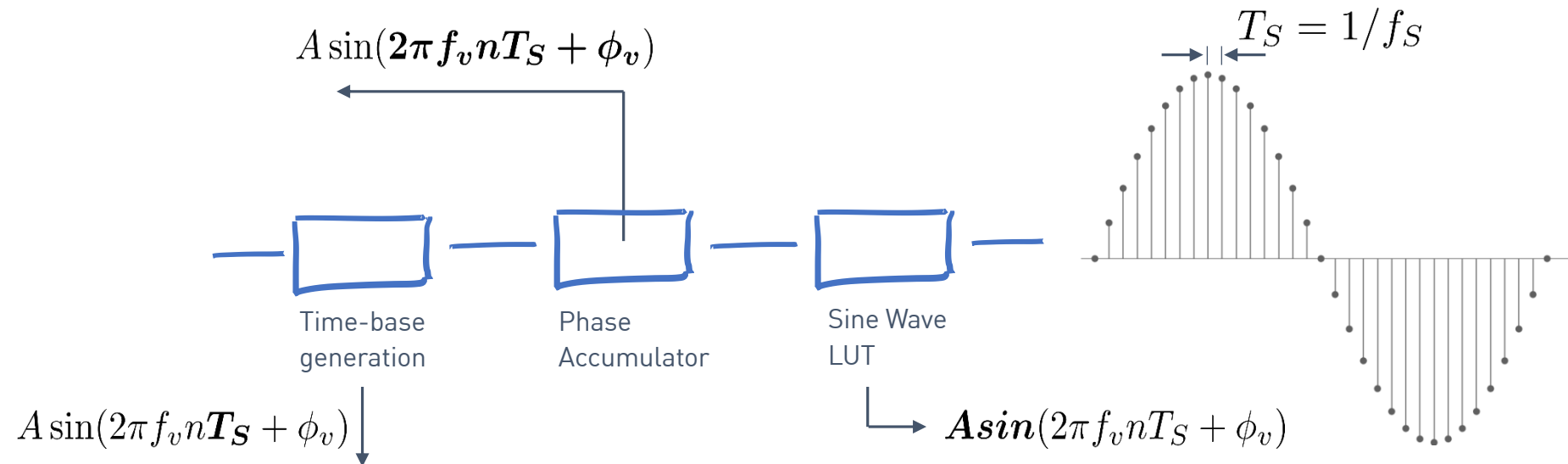
Questions addressed in the following lectures

■ ...



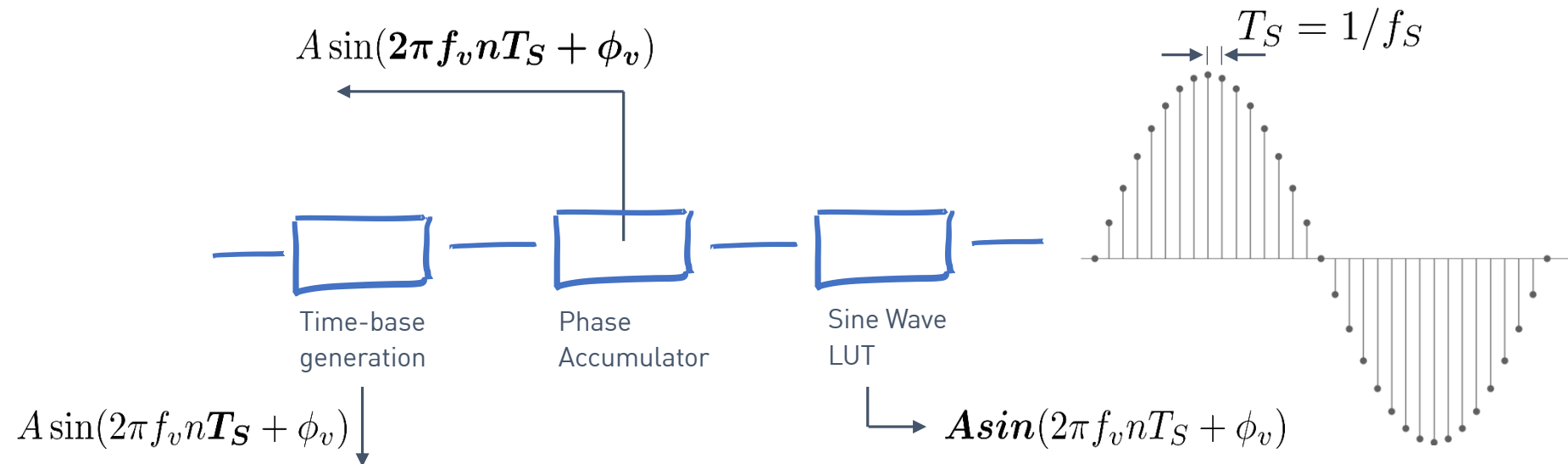
Questions addressed in the following lectures

- How to implement a DDS architecture with variable frequency configuration?



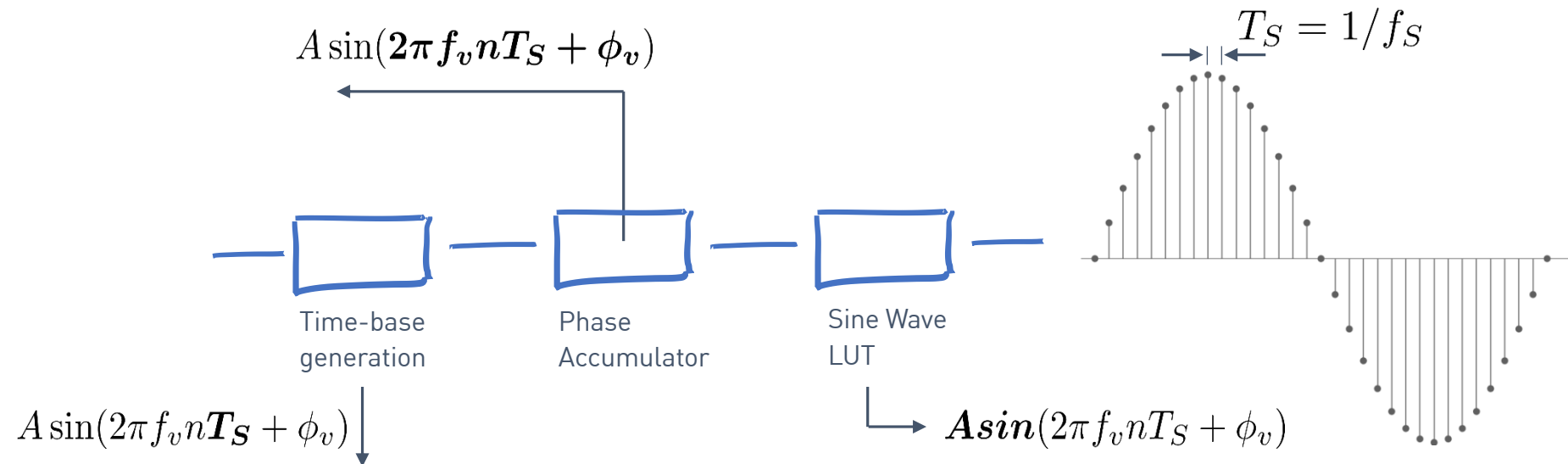
Questions addressed in the following lectures

- How to implement a DDFS architecture with variable frequency configuration?
- How to implement a DDFS architecture with variable phase offset configuration?



Questions addressed in the following lectures

- How to implement a DDFS architecture with variable frequency configuration?
- How to implement a DDFS architecture with variable phase offset configuration?
- How to optimize the memory resource usage?



Direct Digital Frequency Synthesis

- DDFS systems generate periodic, discrete-time waveforms of known frequency. The waveform may be a sinewave. However, it can also be a saw tooth wave, a triangle wave, a square wave, or any other periodic waveform.

Direct Digital Frequency Synthesis

- DDFS systems generate periodic, discrete-time waveforms of known frequency. The waveform may be a sinewave. However, it can also be a saw tooth wave, a triangle wave, a square wave, or any other periodic waveform.
- Direct digital frequency synthesis is a well-established and well-known technique for synthesizing (sinusoidal) signals whenever high frequency resolution, fast changes in frequency and phase, and high spectral purity of the output signal are required.

Direct Digital Frequency Synthesis

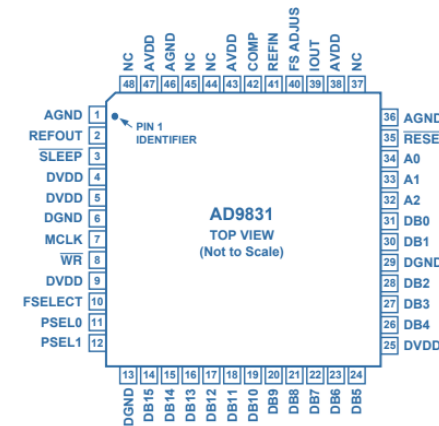
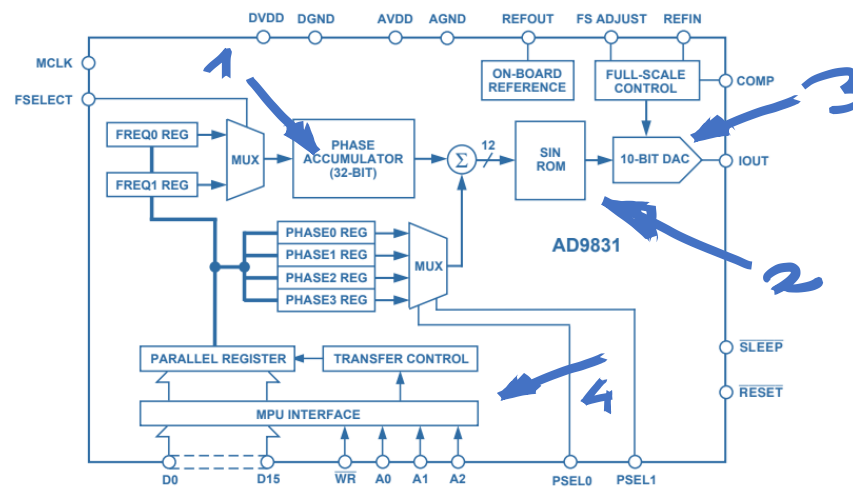
- DDFS systems generate periodic, discrete-time waveforms of known frequency. The waveform may be a sinewave. However, it can also be a saw tooth wave, a triangle wave, a square wave, or any other periodic waveform.
- Direct digital frequency synthesis is a well-established and well-known technique for synthesizing (sinusoidal) signals whenever high frequency resolution, fast changes in frequency and phase, and high spectral purity of the output signal are required.
- It was first described by Tierney et al in 1971.
Tierney, J., Rader, C., and Gold, B., “ A Digital Frequency Synthesizer,” IEEE Transactions on Audio and Electroacoustics, vol. 19, no. 1, March 1971, pp. 48-57.

Direct Digital Frequency Synthesis

- DDFS systems generate periodic, discrete-time waveforms of known frequency. The waveform may be a sinewave. However, it can also be a saw tooth wave, a triangle wave, a square wave, or any other periodic waveform.
- Direct digital frequency synthesis is a well-established and well-known technique for synthesizing (sinusoidal) signals whenever high frequency resolution, fast changes in frequency and phase, and high spectral purity of the output signal are required.
- It was first described by Tierney et al in 1971.
Tierney, J., Rader, C., and Gold, B., “A Digital Frequency Synthesizer,” IEEE Transactions on Audio and Electroacoustics, vol. 19, no. 1, March 1971, pp. 48-57.
- Applications range from measurement instrumentation to modern communications systems, including spread-spectrum and phase shift-keying modulation techniques. A few of them are discussed in the paper “A Technical Tutorial on Digital Signal Synthesis” by Analog Devices.
A Technical Tutorial on Digital Signal Synthesis, available from <https://www.ieee.li/pdf/essay/dds.pdf>

Direct Digital Frequency Synthesis

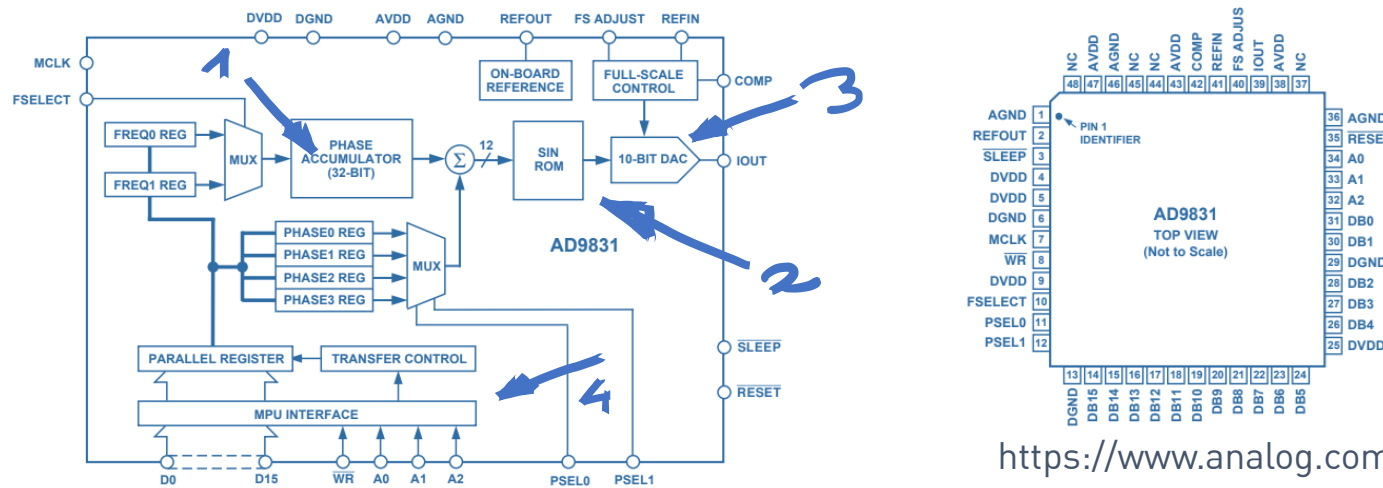
- A popular example of a dedicated DDFS system is shown below. The AD9831 from Analog Devices is a fully integrated DDFS solution employing a phase accumulator (1), a sine look-up table (2) and a 10-bit D/A converter (3) integrated on a single CMOS chip.



<https://www.analog.com/en/products/ad9831.html>

Direct Digital Frequency Synthesis

- A popular example of a dedicated DDFS system is shown below. The AD9831 from Analog Devices is a fully integrated DDFS solution employing a phase accumulator (1), a sine look-up table (2) and a 10-bit D/A converter (3) integrated on a single CMOS chip.



<https://www.analog.com/en/products/ad9831.html>

- Clock rates up to 25 MHz are supported. Frequency accuracy can be controlled to one part in 4 billion. The chip is configured via a dedicated microprocessor interface (4).

Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



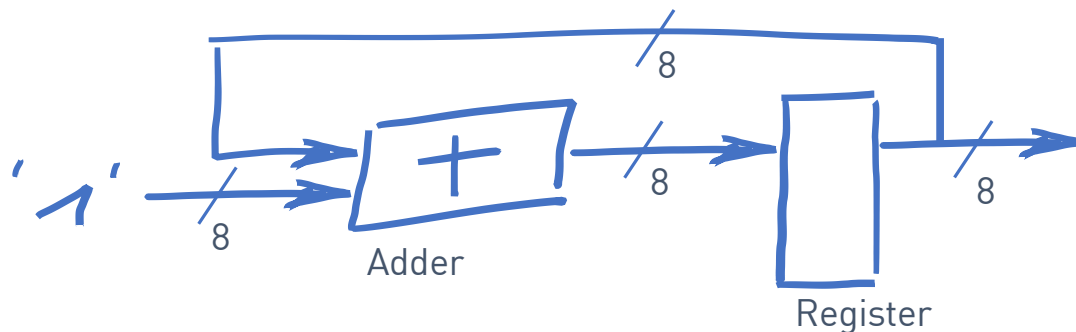
- The phase accumulator (PA) finally generates the phase value used to address the sine wave LUT.
- The phase value is incremented at a rate of determined by the **time base generation unit**.
- The PA itself determines the final frequency of the output signal. The set-up of the previous design used a simple 8-bit counter.

Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$

- The phase accumulator (PA) finally generates the phase value used to address the sine wave LUT.
- The phase value is incremented at a rate of determined by the **time base generation unit**.
- The PA itself determines the final frequency of the output signal. The set-up of the previous design used a simple 8-bit counter.



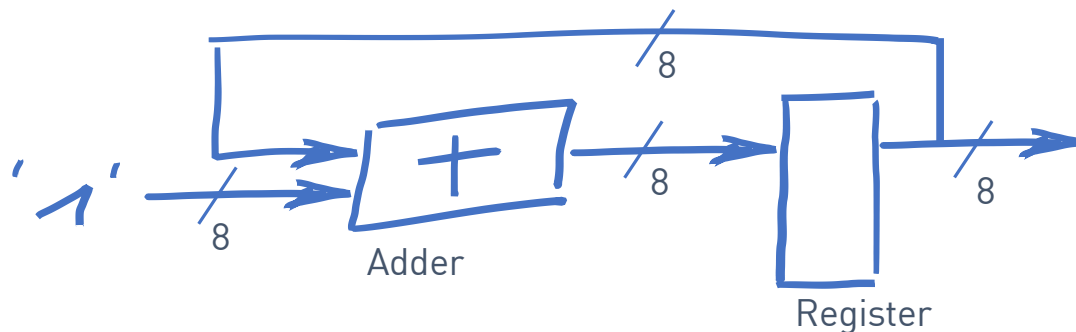
Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



- The phase accumulator (PA) finally generates the phase value used to address the sine wave LUT.
- The phase value is incremented at a rate of determined by the **time base generation unit**.
- The PA itself determines the final frequency of the output signal. The set-up of the previous design used a simple 8-bit counter.
- The counter covers the dec. range from zero to 255 and with that the range from 0° to 360° .
- The phase increment is fixed by a value of $360^\circ/256$



Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



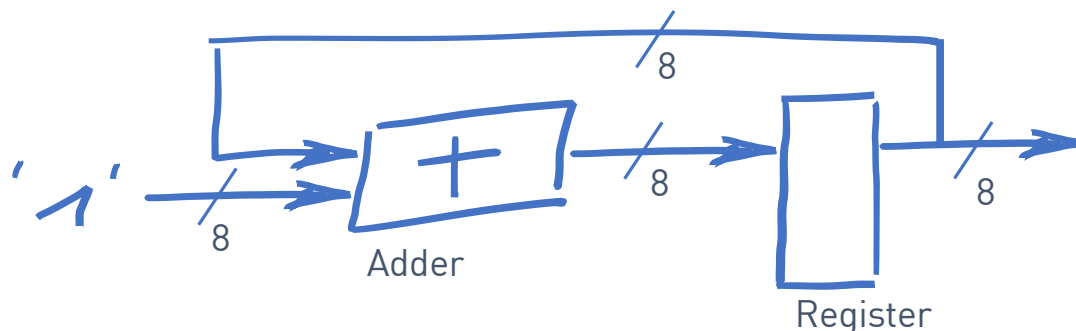
- The phase accumulator (PA) finally generates the phase value used to address the sine wave LUT.
- The phase value is incremented at a rate of determined by the **time base generation unit**.
- The PA itself determines the final frequency of the output signal. The set-up of the previous design used a simple 8-bit counter.

- The counter covers the dec. range from zero to 255 and with that the range from 0° to 360° .
- The phase increment is fixed by a value of $360^\circ/256$
- Let's assume that the time base generation enables the phase accumulator at a rate of 96kHz.
- Thus, the frequency of the output signal is given by:

$$f = 96kHz/256 = 375Hz$$



- This should have been the frequency that you determined by simulating the circuit using Modelsim.



Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



- **Conclusion so far:** The **rate** at which the **address of the LUT changes** is directly proportional to the frequency of the resulting sine wave!

Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



- **Conclusion so far:** The **rate** at which the **address of the LUT changes** is directly proportional to the frequency of the resulting sine wave!
- Keep in mind the LUT address is nothing else than the output of the phase accumulator, i.e. the current phase value ...

Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



- Conclusion so far:** The **rate** at which the **address of the LUT changes** is directly proportional to the frequency of the resulting sine wave!
- Keep in mind the LUT address is nothing else than the output of the phase accumulator, i.e. the current phase value ...
- Consider the following scenario:** By changing the setting of the time base generation unit to 192kHz, we are increasing the rate at which we read from the LUT memory.
- Therefore, the frequency of the output signal becomes:

$$f = 192kHz / 256 = 750Hz$$



Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



- **Conclusion so far:** The **rate** at which the **address of the LUT changes** is directly proportional to the frequency of the resulting sine wave!
 - Keep in mind the LUT address is nothing else than the output of the phase accumulator, i.e. the current phase value ...
 - **Consider the following scenario:** By changing the setting of the time base generation unit to 192kHz, we are increasing the rate at which we read from the LUT memory.
 - Therefore, the frequency of the output signal becomes:
- So, the frequency of the sine wave is obviously directly proportional to the system clock generated by the time base generation unit.

$$f = 192kHz / 256 = 750Hz$$



Direct Digital Frequency Synthesis

- However, changing the system frequency is to change the output frequency is a rather poor approach ...

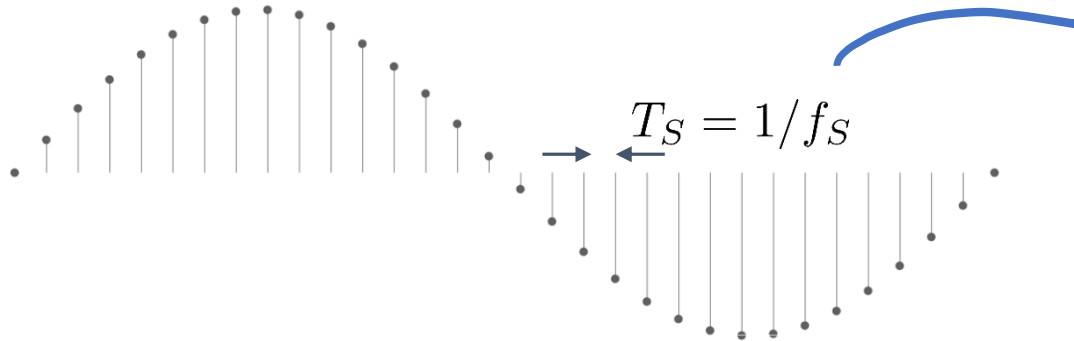
Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



- Let's address the following question: How to double the output frequency by keeping the system frequency at 96kHz?



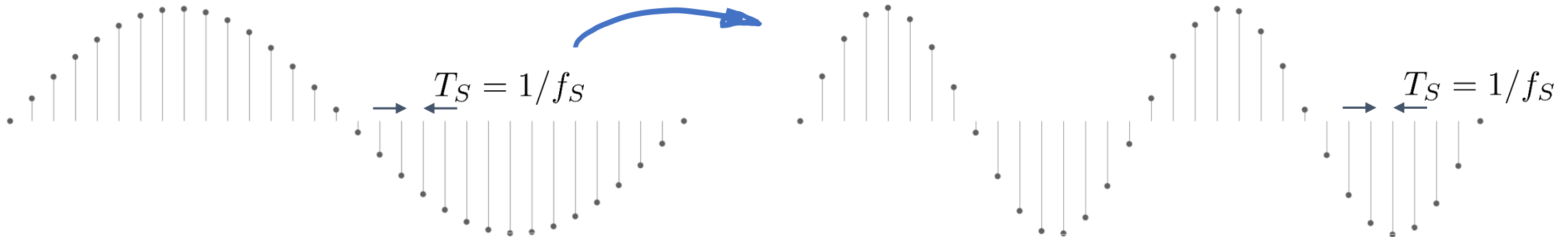
Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$

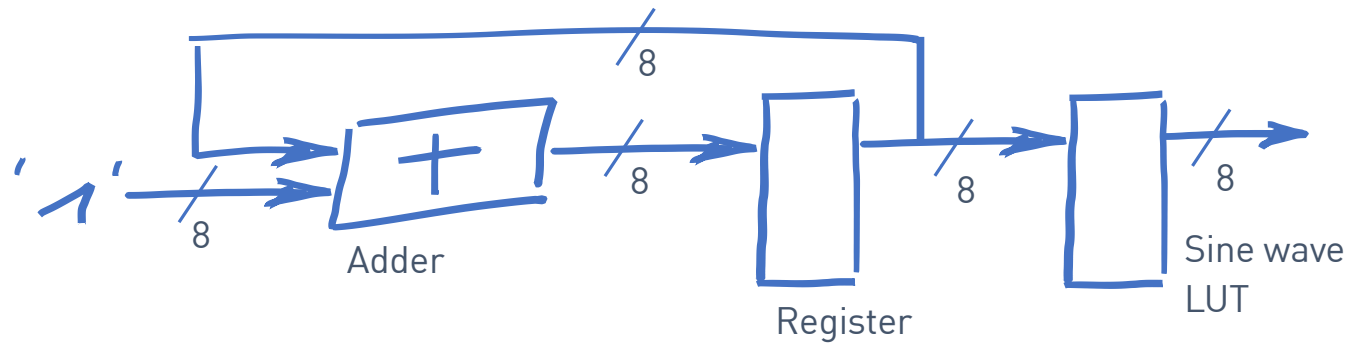


- Let's address the following question: How to double the output frequency by keeping the system frequency at 96kHz?



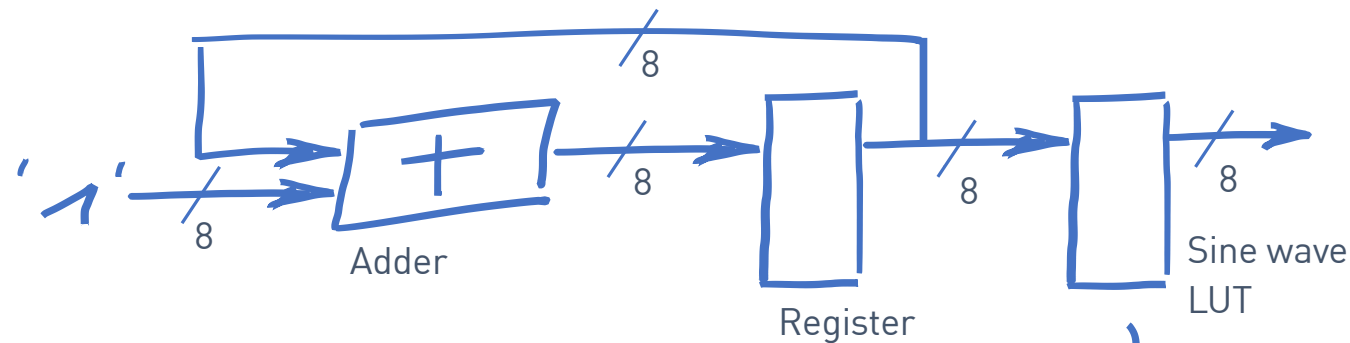
Direct Digital Frequency Synthesis

- An obvious solution would be given by reprogramming the LUT.

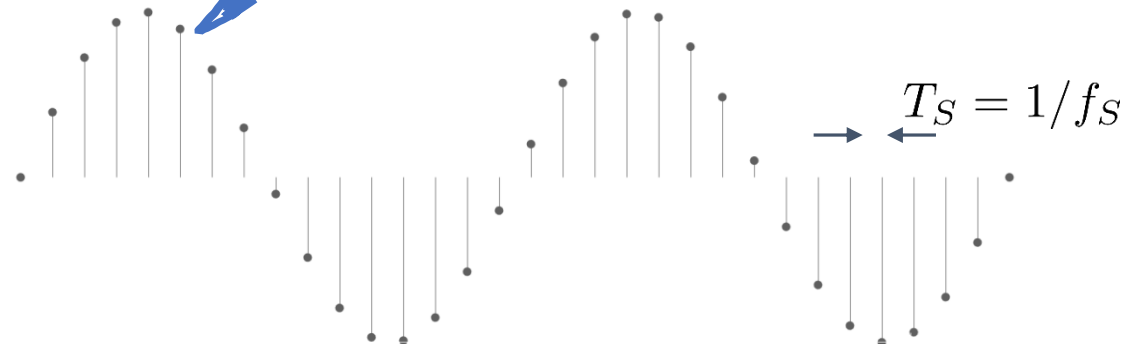


Direct Digital Frequency Synthesis

- An obvious solution would be given by reprogramming the LUT.

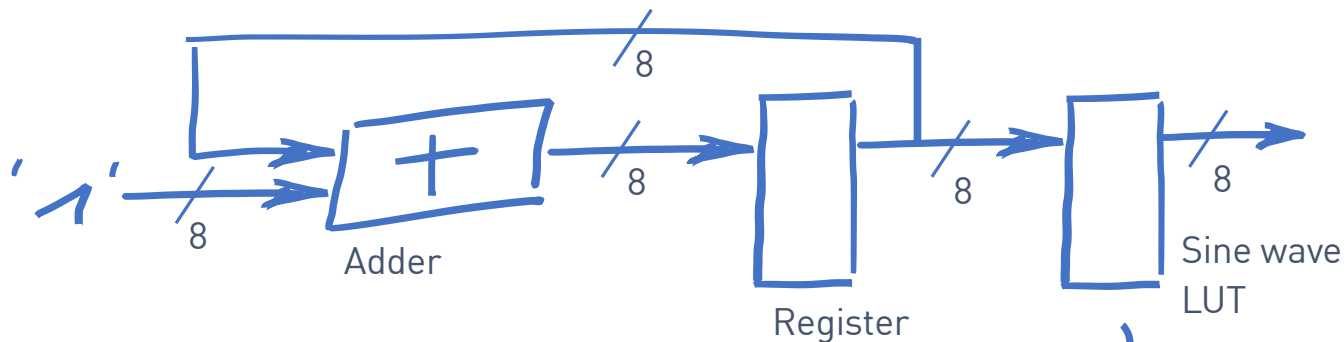


- Instead of storing a single sine wave period using 256 values, we could simply map two periods on the available 256 memory cells.

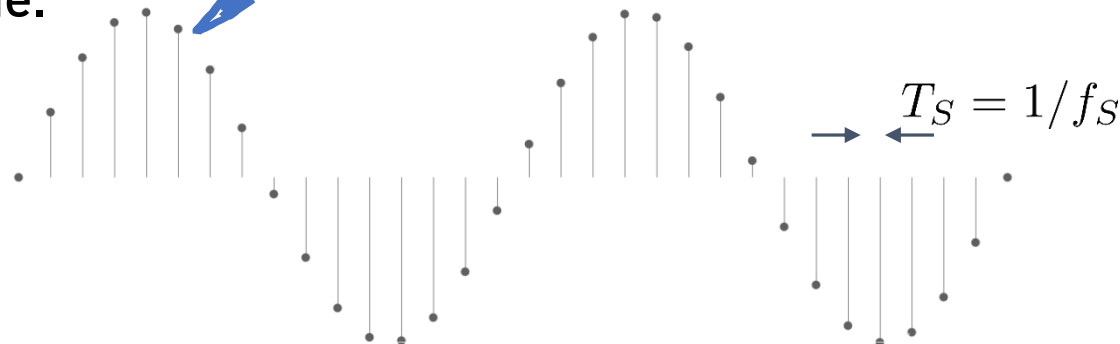


Direct Digital Frequency Synthesis

- An obvious solution would be given by reprogramming the LUT.

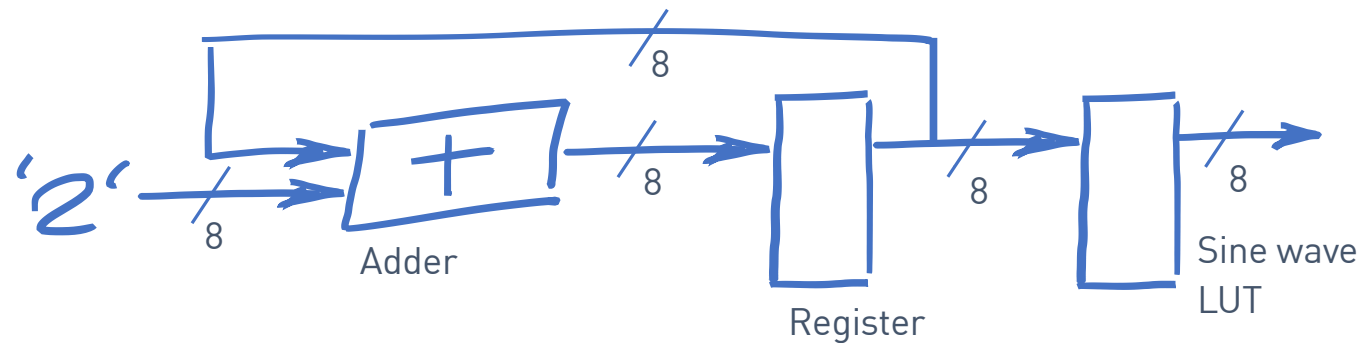


- Instead of storing a single sine wave period using 256 values, we could simply map two periods on the available 256 memory cells.
- However, this approach is rather inflexible. **Changing the frequency, would always require a full memory update at run-time.**



Direct Digital Frequency Synthesis

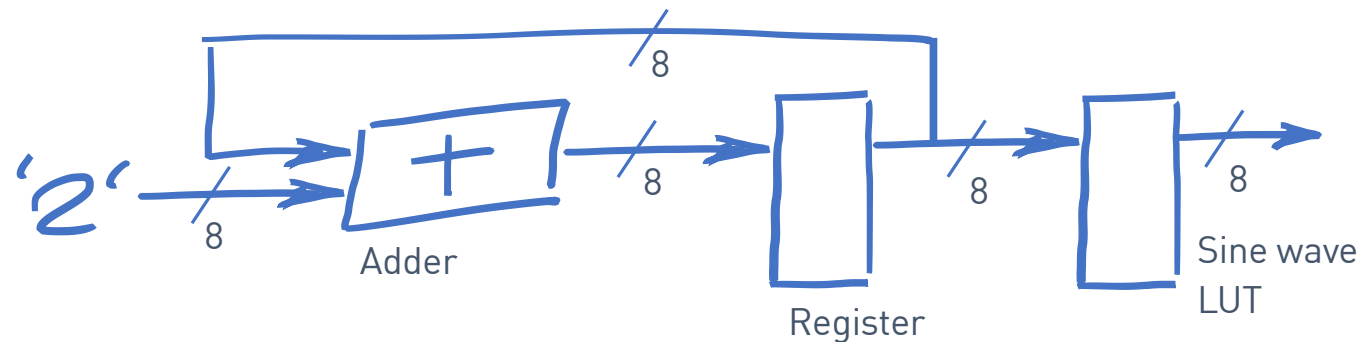
- A more reasonable solution is to **alter the phase increment** itself.



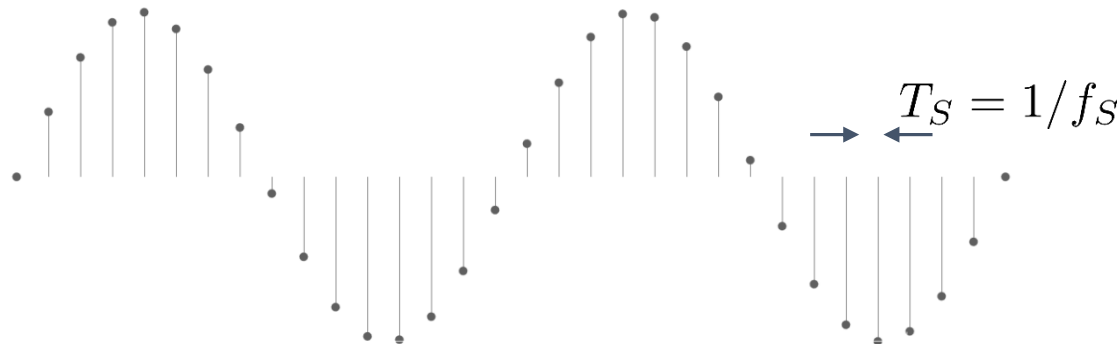
- Instead of incrementing the phase accumulator by '1', let's increment it by '2'.

Direct Digital Frequency Synthesis

- A more reasonable solution is to alter the phase increment itself.

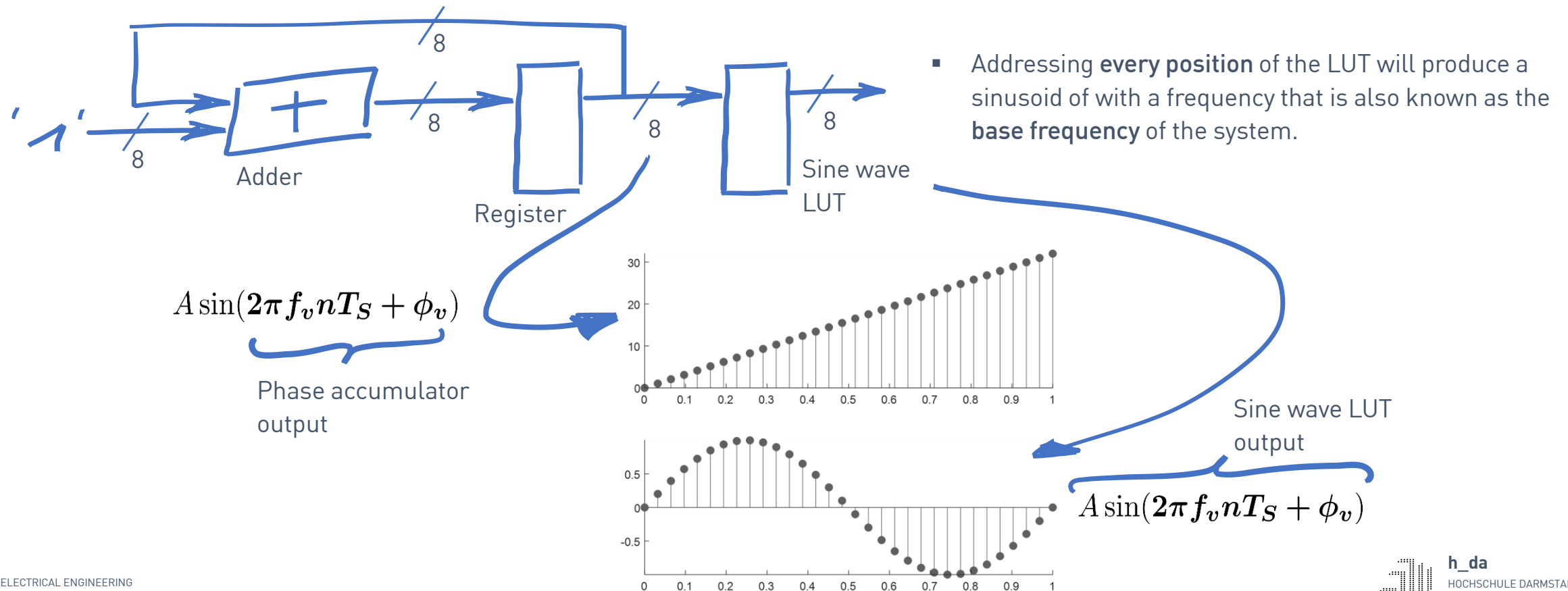


- Instead of incrementing the phase accumulator by '1', let's increment it by '2'.
- By doing so, we only address every second memory cell of the LUT and thus, double the frequency of the output signal ...



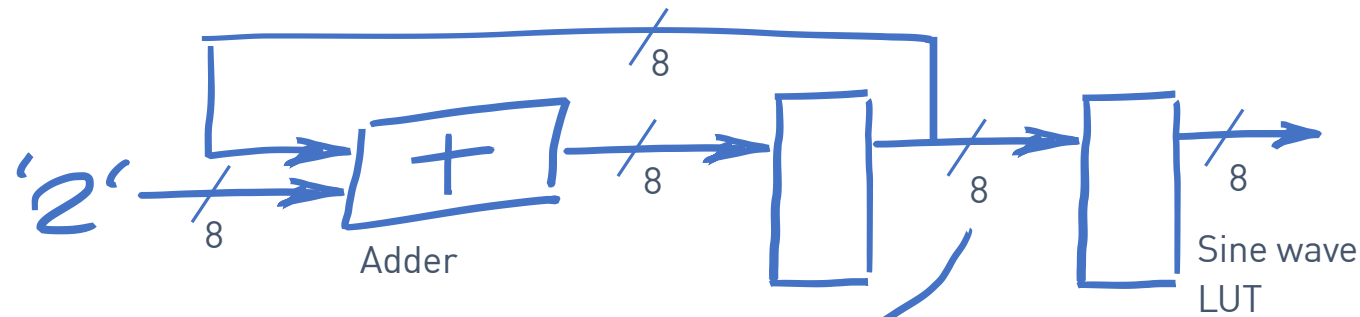
Direct Digital Frequency Synthesis

- Incrementing the phase accumulator by ,1‘



Direct Digital Frequency Synthesis

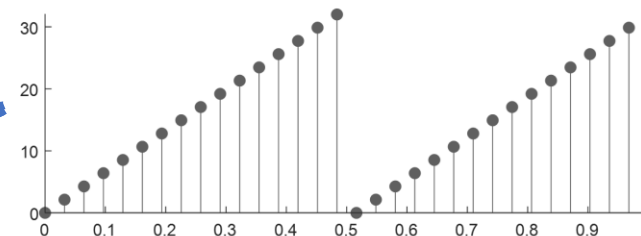
- Incrementing the phase accumulator by 2^8



- Addressing **only every second position** of the LUT will produce a sinusoid of twice the frequency of addressing every single position of the sine LUT! **Or in other words:** A frequency of twice the base frequency

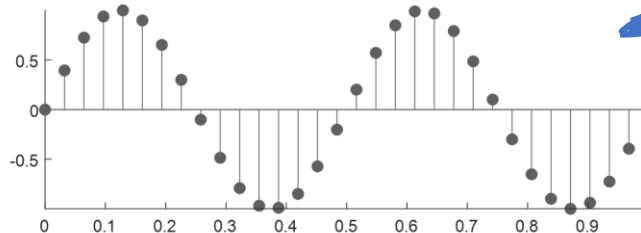
$$A \sin(2\pi f_v n T_S + \phi_v)$$

Phase accumulator output



Sine wave LUT output

$$A \sin(2\pi f_v n T_S + \phi_v)$$



Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



- **Conclusion so far:** A larger increment at each update to the phase accumulator will mean that the phase reaches the full cycle value faster and the frequency is correspondingly high.
- Keep in mind that the phase value is bounded

$$\phi = (2\pi f_v n T_S + \phi_v) \text{ rad (mod } 2\pi)$$

and wraps every 2π radians = 1 cycle ...

Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



- A phase accumulator with variable frequency support is realized by providing a configurable phase increment.
- Due to the fact that the phase increment determines the frequency of the output signal, it is also denoted as the frequency tuning word **FTW**.

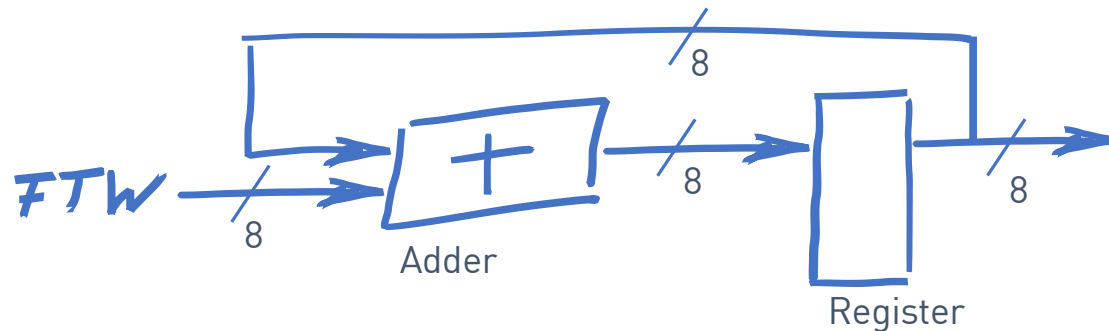
Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

$$A \sin(2\pi f_v n T_S + \phi_v)$$



- A phase accumulator with variable frequency support is realized by providing a configurable phase increment.
- Due to the fact that the phase increment determines the frequency of the output signal, it is also denoted as the frequency tuning word **FTW**.



- Let's find a general expression that describes the DDFS output frequency for a given FTW and LUT configuration:

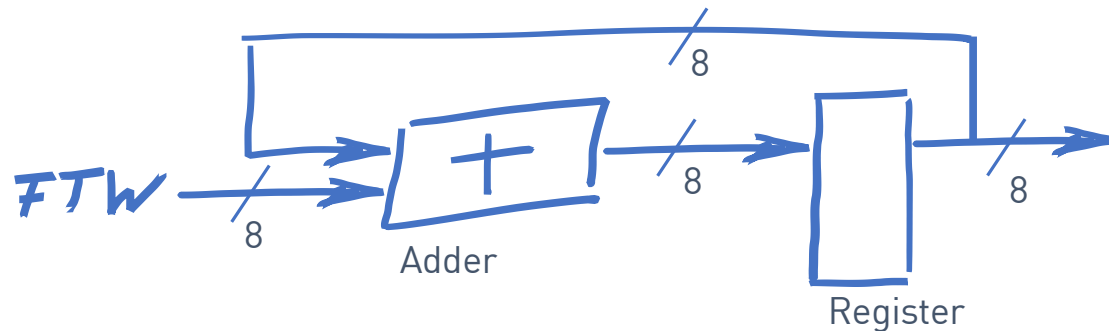
Direct Digital Frequency Synthesis

- How to implement a **Phase accumulator** with variable frequency support?

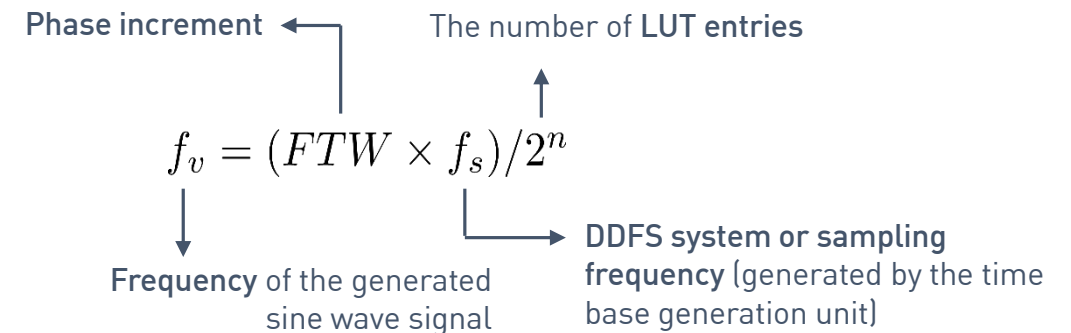
$$A \sin(2\pi f_v n T_s + \phi_v)$$



- A phase accumulator with variable frequency support is realized by providing a configurable phase increment.
- Due to the fact that the phase increment determines the frequency of the output signal, it is also denoted as the frequency tuning word **FTW**.



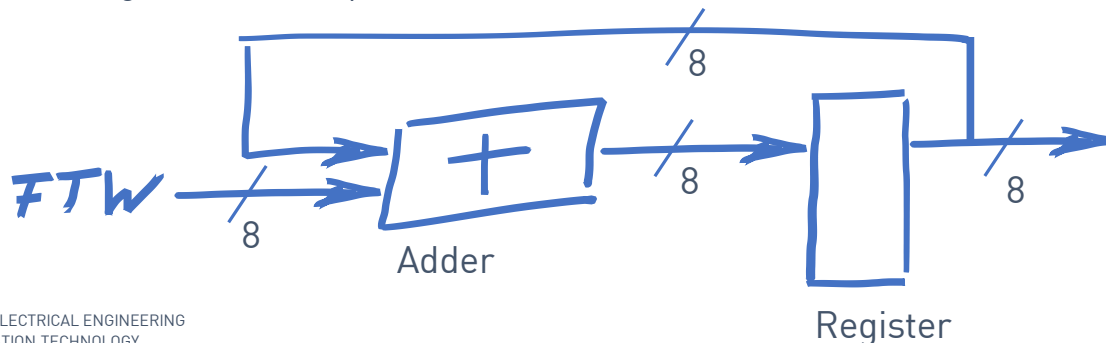
- Let's find a general expression that describes the DDFS output frequency for a given FTW and LUT configuration:



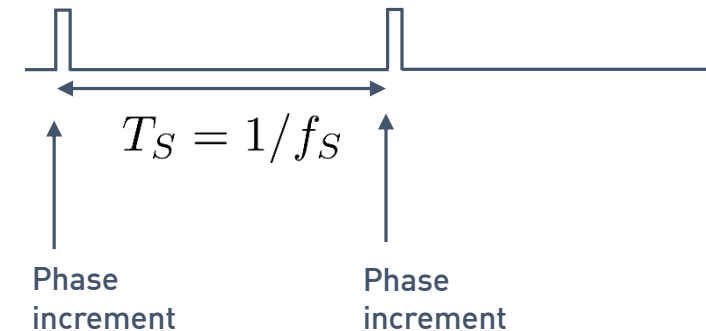
Direct Digital Frequency Synthesis

```
1. module phase_acc (  
2.     input logic clk, reset_n, enable,  
3.     input logic [7:0] ftw, output logic [7:0] q  
4. );  
5.  
6.     always_ff@(posedge clk)  
7.         if(reset_n == 1'b0)  
8.             q <= '0;  
9.         else if(enable)  
10.            q <= q + ftw;  
11.  
12. endmodule
```

[SystemVerilog] Source Code: phase_acc.sv



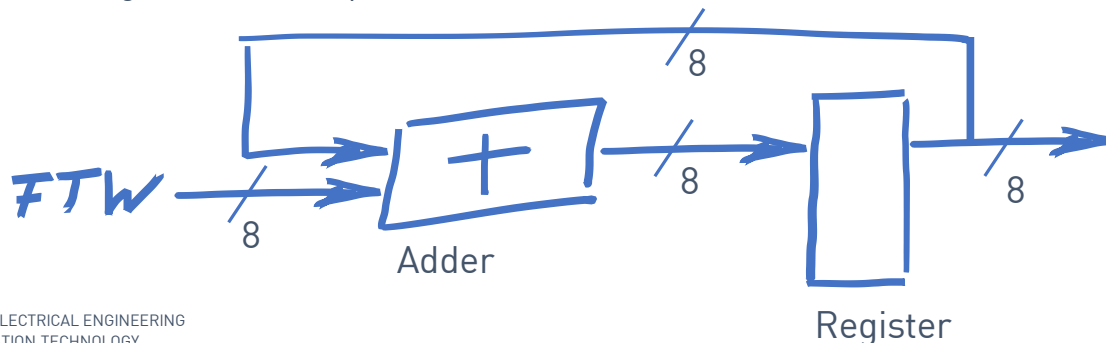
- The phase accumulator is now incremented by the respective FTW word chosen.



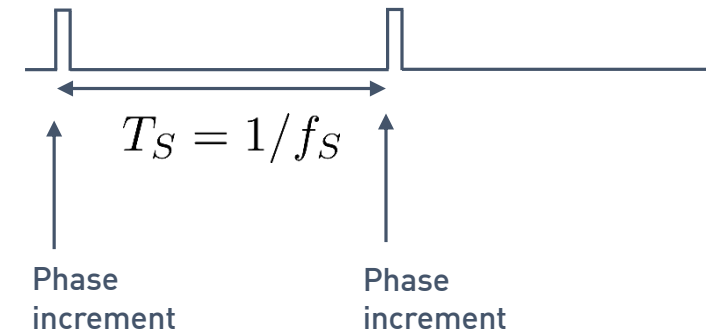
Direct Digital Frequency Synthesis

```
1. module phase_acc (  
2.     input logic clk, reset_n, enable,  
3.     input logic [7:0] ftw, output logic [7:0] q  
4. );  
5.  
6.     always_ff@(posedge clk)  
7.         if(reset_n == 1'b0)  
8.             q <= '0;  
9.         else if(enable)  
10.            q <= q + ftw;  
11.  
12. endmodule
```

[SystemVerilog] Source Code: phase_acc.sv



- The phase accumulator is now incremented by the respective FTW word chosen.
- In theory, the maximum output frequency is half the system (or sampling) frequency of the DDFS architecture (In reality, we chose output values below that reference).
- Therefore, FTW values only range from 0 to 2^{n-1} and cover only the lower half of available phase bits.



Prof describes about this exercise at 56min of audio recoding.

Direct Digital Frequency Synthesis

- **Exercise:** Assume a DDS architecture with an 8-bit phase accumulator operating at a system (or sampling) frequency of 10MHz. All bits of the phase accumulator are used to index (or address) the LUT that contains a full period of a sine wave.
- a) Sketch the block diagram of the DDS architecture (clearly state all relevant parameters like bit width, address width, ...) and determine the phase resolution for the present circuit configuration in degrees!
- b) Determine the frequency tuning word (FTW) in order to synthesise a frequency of 2.4MHz.
- c) Apply the FTW value calculated in the previous problem to the present circuit configuration. Finally, determine the actual frequency synthesised.
- d) What is the resulting absolute or relative error?
- e) Implement the system using SystemVerilog.
- f) Create a dedicated testbench to verify the correct functionality of the ddfs system implemented by simulating the design using Mentor Modelsim.

Direct Digital Frequency Synthesis

- The concepts developed so far covers an output frequency range starting with the base frequency (which is equivalent to a FTW of '1') up to a frequency set by a FTW of 2^{n-1} .

Direct Digital Frequency Synthesis

- The concepts developed so far covers an output frequency range starting with the base frequency (which is equivalent to a FTW of '1') up to a frequency set by a FTW of 2^{n-1} .
- Moreover, the frequency resolution of the current architecture is limited to

$$\Delta f_v = f_s / 2^n$$

- For a DDFS architecture with an 8-bit phase accumulator operating at a system (or sampling) frequency of 10MHz, the frequency resolution is given by 39,0625kHz.

Direct Digital Frequency Synthesis

- The concepts developed so far covers an output frequency range starting with the base frequency (which is equivalent to a FTW of '1') up to a frequency set by a FTW of 2^{n-1} .
- Moreover, the frequency resolution of the current architecture is limited to

$$\Delta f_v = f_s / 2^n$$

- For a DDFS architecture with an 8-bit phase accumulator operating at a system (or sampling) frequency of 10MHz, the frequency resolution is given by 39,0625kHz.
- Therefore, it is only possible to generate frequencies which are integer multiples of that 39,0625kHz.

Embedded Signal Processing Systems

h_da – fbeit - IMSE – Embedded Signal Processing

Based on these observation, let's address the following questions

- ...

Based on these observation, let's address the following questions

- How to generate frequencies smaller than the actual base frequency ? (given by a frequency tuning word FTW of '1').

Based on these observation, let's address the following questions

- How to generate frequencies smaller than the actual base frequency ? (given by a frequency tuning word FTW of '1').
- How to further increase the frequency resolution of a DDFS system?

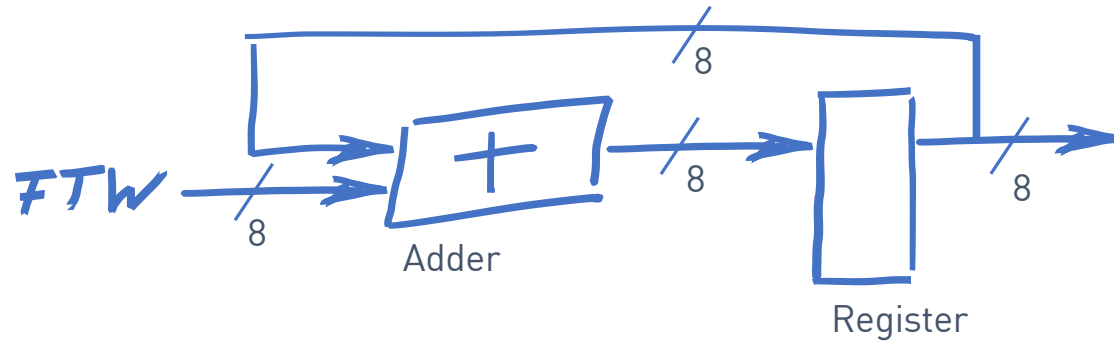
Based on these observation, let's address the following questions

- How to generate frequencies smaller than the actual base frequency ? (given by a frequency tuning word FTW of '1').
- How to further increase the frequency resolution of a DDFS system?
- Both questions are addressed by a extending the current phase accumulator structure ...



Direct Digital Frequency Synthesis

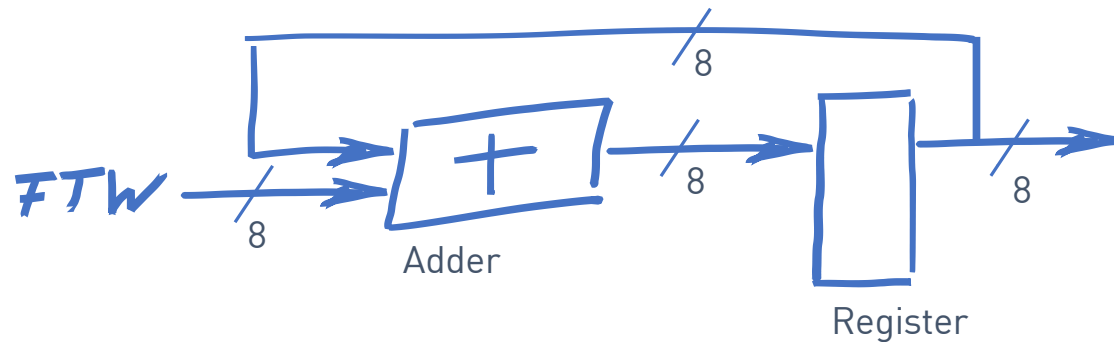
- The architecture so far was an **integer-only accumulator**



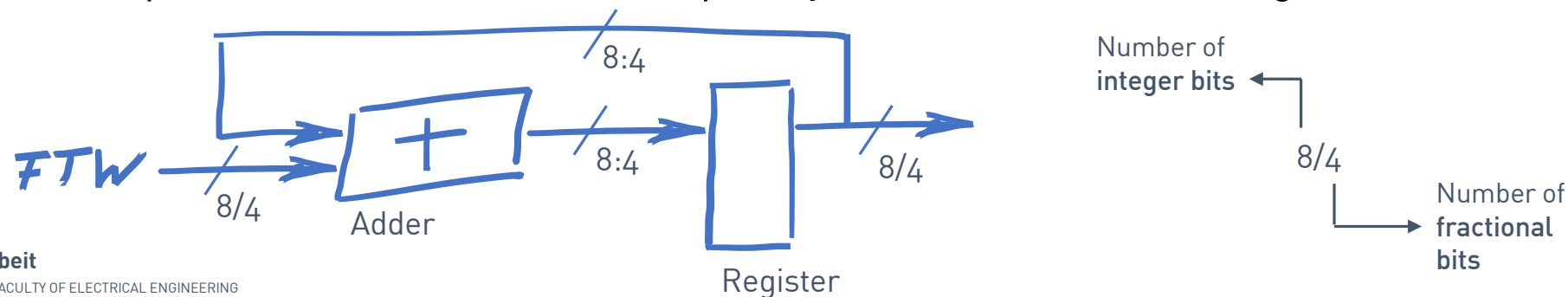
▪ :

Direct Digital Frequency Synthesis

- The architecture so far was an integer-only accumulator

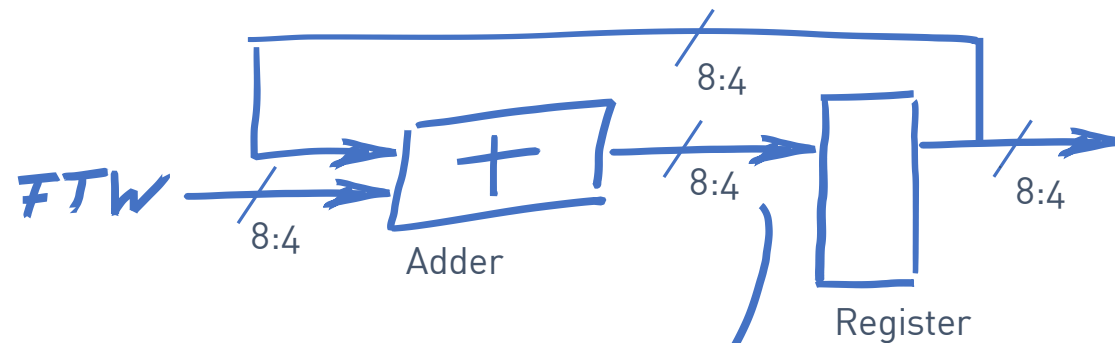


- The frequency resolution can be significantly increased by extending the present phase accumulator with a fractional part. This also offers the possibility to synthesize frequencies below the base frequency. Consider the following structure:

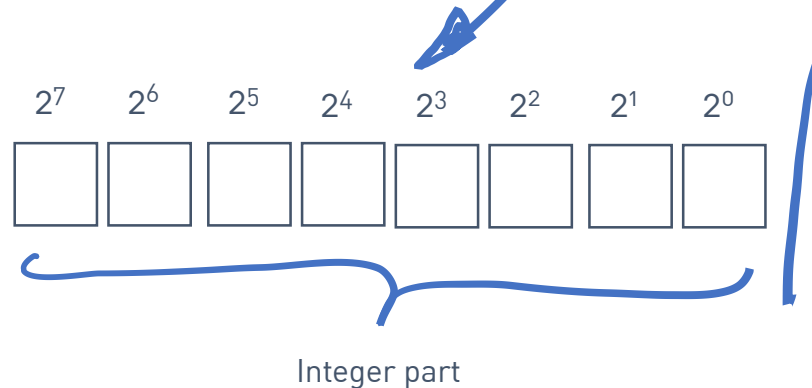


Direct Digital Frequency Synthesis

- Let's have a closer look at the structure of a phase word.

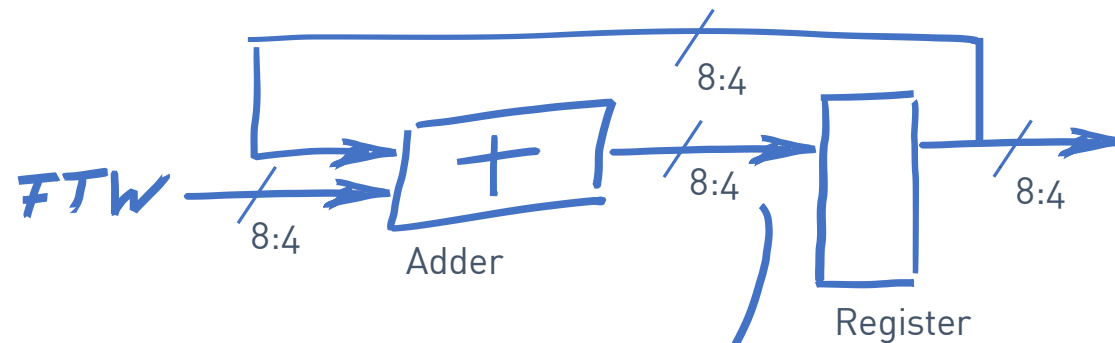


- As in the previous configuration, we still have 8 integer bits

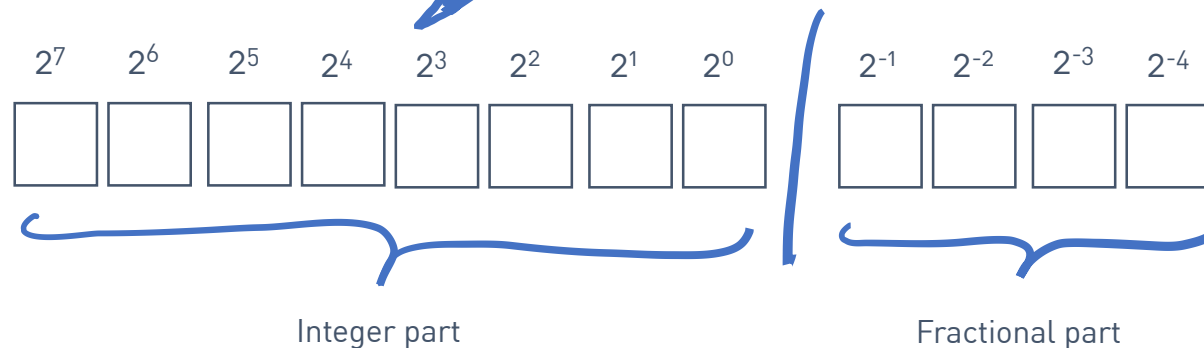


Direct Digital Frequency Synthesis

- Let's have a closer look at the structure of a phase word.

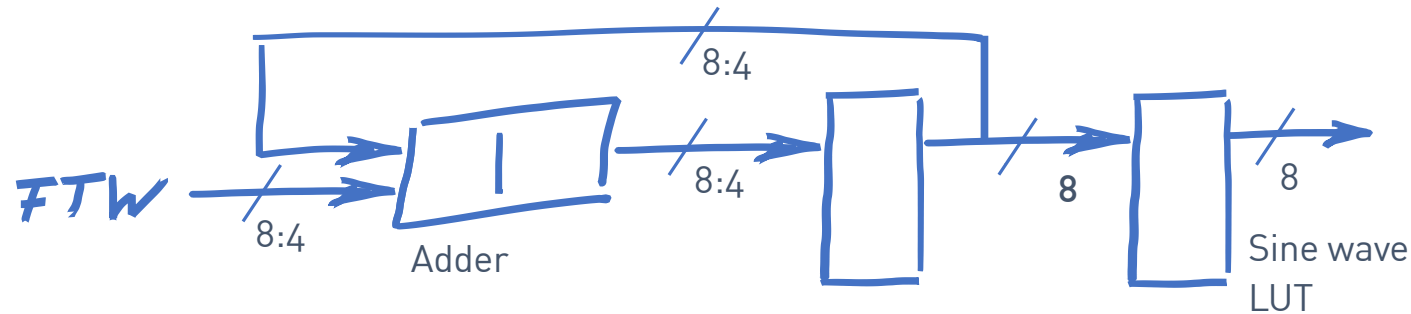


- As in the previous configuration, we still have 8 integer bits, but right now with 4 additional fractional bits on the LSB side

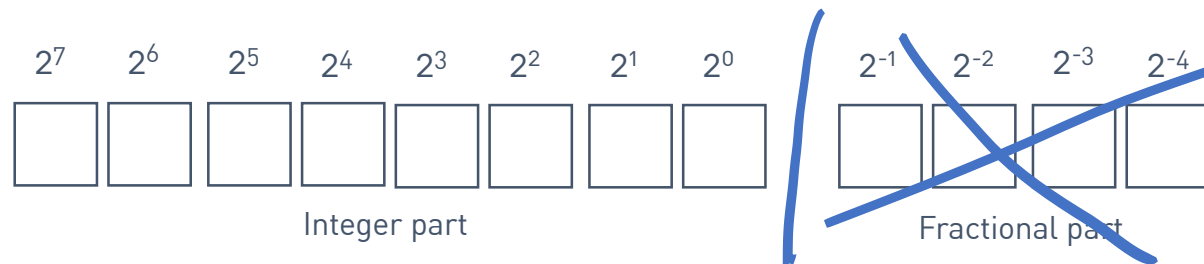


Direct Digital Frequency Synthesis

- When addressing the look-up table, we are now using only the whole part of the accumulator output.

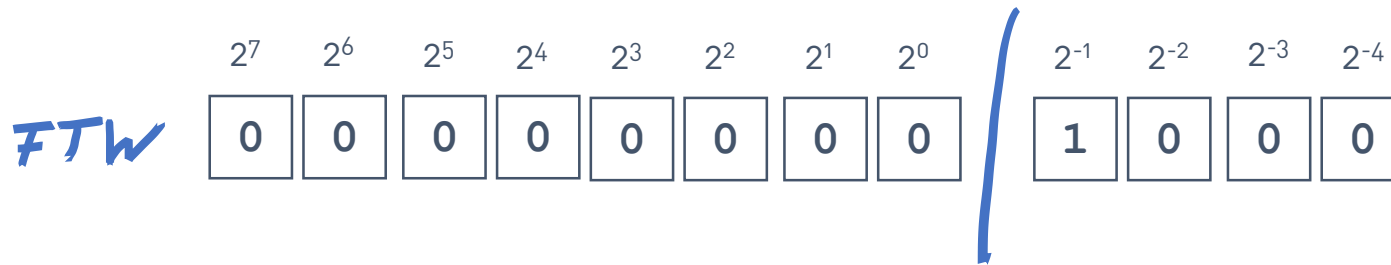


- The fractional bits are disregarded.



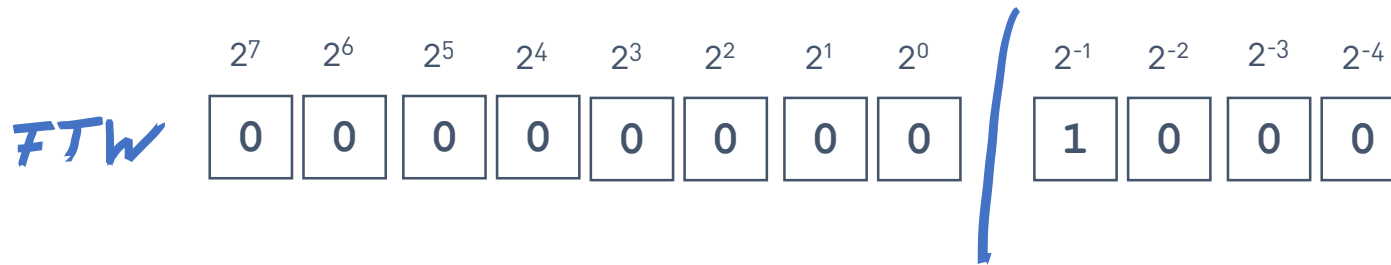
Direct Digital Frequency Synthesis

- Consider the following frequency tuning word FTW



Direct Digital Frequency Synthesis

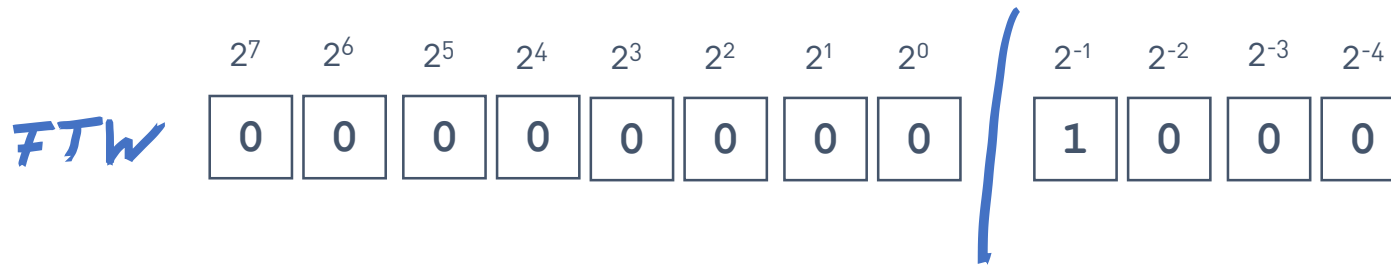
- Consider the following frequency tuning word FTW



- Using this phase increment, the LUT address only changes every second clock cycle.

Direct Digital Frequency Synthesis

- Consider the following frequency tuning word FTW



- Using this phase increment, the LUT address only changes every second clock cycle.
- By that, the base frequency is halved by a factor of 2

Direct Digital Frequency Synthesis

- Moreover, the introduction of a fractional part also impacts the final frequency resolution.

Direct Digital Frequency Synthesis

- Moreover, the introduction of a fractional part also impacts the final frequency resolution.
- If we have a fractional part of b bits, the minimum phase increment or step size can be determined as follows:

$$\Delta FTW = 1/2^b$$

- This is also denoted as the FTW resolution.

Direct Digital Frequency Synthesis

- The FTW (or step size) resolution translates directly into the frequency resolution of the synthesized sine wave, and may be represented by:

$$\Delta f_v = \Delta FTW \times f_s / N = f_s / (2^b N)$$

Direct Digital Frequency Synthesis

- **Exercise:** Assume a DDFS system with a 256-entry LUT and a system frequency of 10MHz. Determine the FTW resolution as well as the DDFS output frequency resolution for the following system configurations.



- a) Phase accumulator with 0 fractional bits
- b) Phase accumulator with 1 fractional bits
- c) Phase accumulator with 2 fractional bits
- d) Phase accumulator with 5 fractional bits
- e) Phase accumulator with 10 fractional bits
- f) Phase accumulator with 20 fractional bits

Direct Digital Frequency Synthesis

- **Exercise:** Assume a DDFS architecture with a 12-bit phase accumulator operating at a system frequency of 10MHz. Only the eight most-significant bits of the accumulator are used to index the LUT, which contains a full sine wave. .
- a) Sketch the block diagram of the DDFS architecture (clearly state all relevant parameters like bit width, address width, ...) and determine the phase resolution for the present circuit configuration in degrees!
- b) Determine the frequency tuning word (FTW) in order to synthesise a frequency of 2.4MHz.
- c) Apply the FTW value calculated in the previous problem to the present circuit configuration. Finally, determine the actual frequency synthesised.
- d) What is the resulting absolute or relative error?
- e) Compare the performance of the present DDFS system with the previous architecture configuration (integer only accumulator) in terms of their ability to precisely synthesise a certain frequency (use the value of 2.4MHz for your discussion).
- f) Implement the system using SystemVerilog.
- g) Create a dedicated testbench to verify the correct functionality of the ddfs system implemented by simulating the design using Mentor Modelsim.

Lab@Home

Using Mentor Modelsim to simulate the DDFS design

- Open a console and create a project directory

```
mkdir ddfs_project
```

- Use your favourite editor to create the previous design. Make sure that all source files are stored in the project directory. For example:

```
gedit time_base_gen.sv &
```

- After creating all files including the testbench, start Modelsim

```
vsim &
```

- Start a new project using

```
File ... New ... project
```

- In the dialog box sepecify the name and project location and press OK
- In the next popup window, Add existing File, and select

```
ddfs_tb.sv
```

Lab@Home

Using Mentor Modelsim to simulate the DDFS design

- Next, we compile the design by

`Compile using Compile...Compile All`

- You should see a message that states `Compile of ddfs_tb.sv was successful.`

- In the next step, we simulate the design. Choose

`Simulate ... Start Simulation`

- In the dialog box navigate to `work ... ddfs_tb.sv`

- You should see the testbench signal names in a panel. Select them all and choose

`Add ... To Wave ... Selected Signals`

- Finally, perform

`Simulate ... Run ... Run -All.`

- You should see waveforms in the right panel.

- Right-click the name of the `tb_q` and choose `Format...Analog(automatic)`

Embedded Signal Processing Systems

h_da – fbeit - IMSE – Embedded Signal Processing

Lab@Home

- You can use the Intel Quartus Lite toolchain as an alternative

DDFS Architectures

Today's Agenda

Additional exercises on DDFS based system design

1. General aspects about DDFS systems.
2. Derivation of the DDFS frequency equation.
3. Memory optimization in DDFS based systems.
4. DDFS systems with variable phase offsets.
5. DDFS based I/Q signal generation.
6. DDFS based Chirp signal generation.

Lab@Home: Practical Implementation of DDFS Architectures

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Practical Implementation of DDFS Architectures

- **[Exercise]:** Shortly explain the following points:
 - a) What is determining the phase resolution in a DDFS system?
 - b) What is determining the amplitude resolution in a DDFS system?
 - c) What is determining the frequency resolution in a DDFS system?

Practical Implementation of DDFS Architectures

- **[Exercise]:** The output frequency of a DDFS based signal generation scheme for a given frequency tuning word FTW is calculated as follows:

$$f_v = (FTW \times f_s) / 2^n$$

Phase increment

The number of LUT entries

Frequency of the generated sine wave signal

DDFS system or sampling frequency (generated by the time base generation unit)

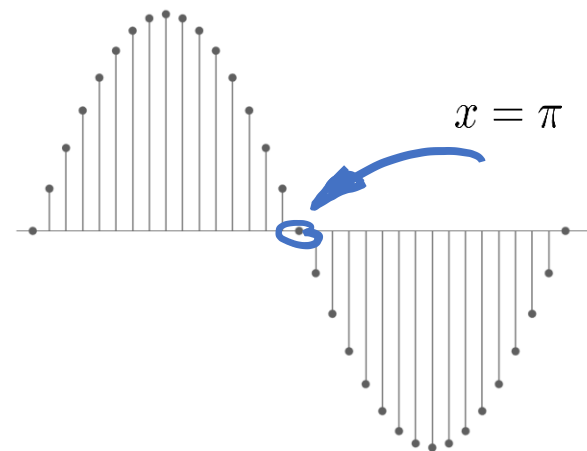
- This equation has been introduced in lecture '3', however without clearly discussing its actual origin.

- a) The aim of this exercise is to derive the upper equation. It is helpful to use the general definition of frequency as a starting point.

Practical Implementation of DDFS Architectures

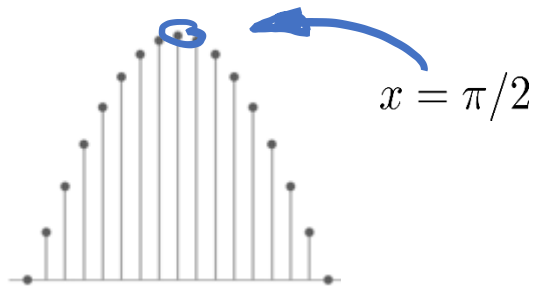
- The DDFS designs so far have been solely based on a LUT which contains a full sine wave cycle. In addition to the periodicity and modulo- 2π characteristic, the symmetric properties of the sine wave function $\sin(x)$ can be used to further reduce the amount of memory resources required. Given the fact that the sine wave function is antisymmetric about the point $x = \pi$, $[0, 2\pi]$.
- Exploiting this antisymmetric nature of the sine wave function, it is possible to use the following relationship:

$$\sin(x + \pi) = -\sin(x), \quad [0, \pi]$$



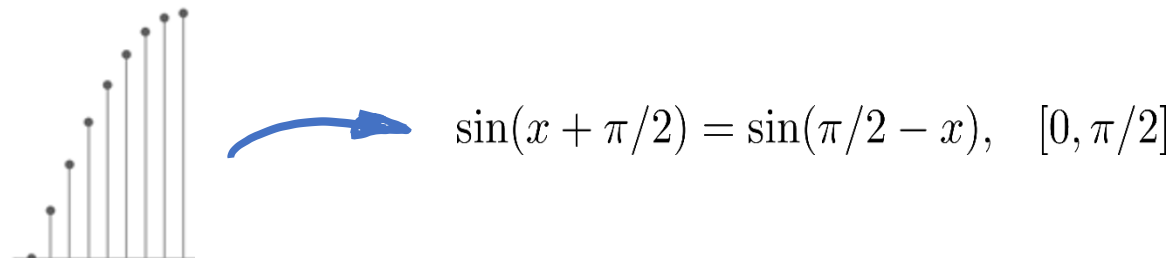
Practical Implementation of DDFS Architectures

- In the same way, the sine wave function $\sin(x)$ shows a symmetric behavior around the point $x = \pi/2$, $[0, \pi]$ so that the following relationship can be used $\sin(x + \pi/2) = \sin(\pi/2 - x)$, $[0, \pi/2]$.



- In conclusion, it is only necessary to store the first quadrant[†] of a sine wave within the LUT and to apply dedicated pre- and post-processing operations:

[†] it's even possible to go below that

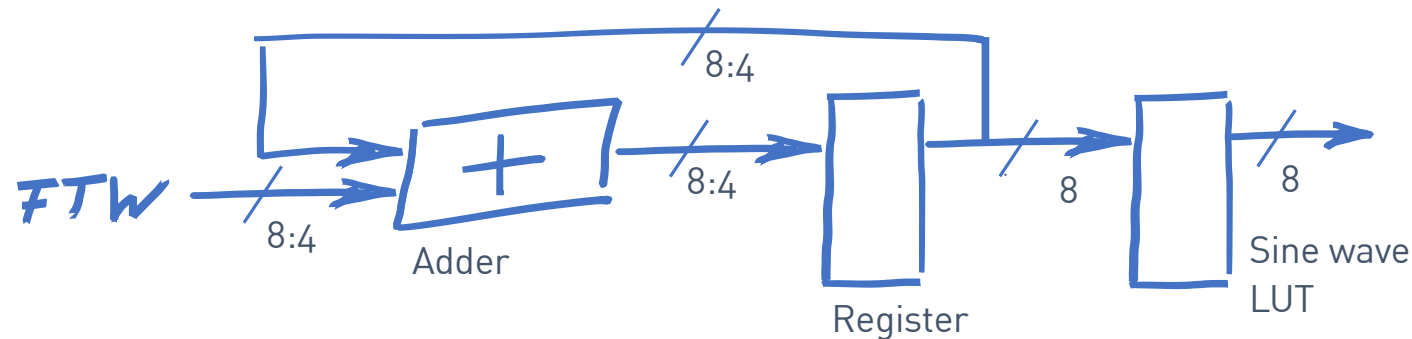


Practical Implementation of DDFS Architectures

- **[Exercise]:** Assume a DDFS architecture that exploits the relation $\sin(x + \pi/2) = \sin(\pi/2 - x)$ to reduce the sine wave LUT memory footprint.
 - a) Implement the system using SystemVerilog.
 - b) Create a dedicated testbench to verify the correct functionality of the DDFS system implemented by simulating the design using Mentor Modelsim.

Practical Implementation of DDFS Architectures

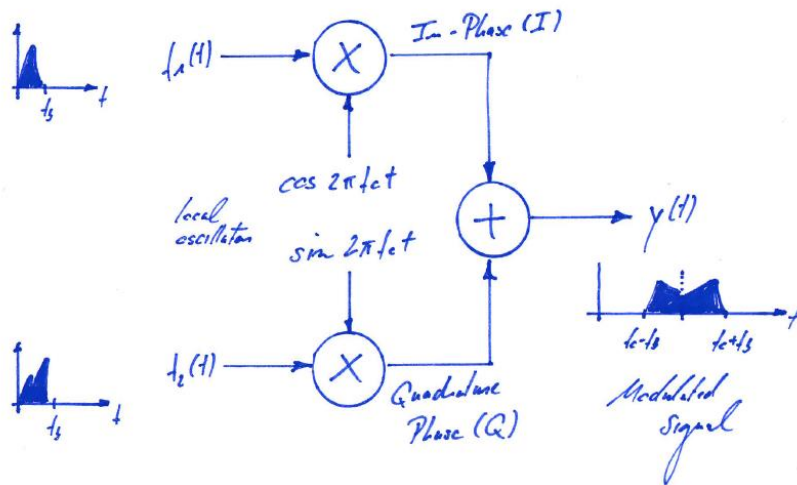
- **[Exercise]:** Assume a DDFS architecture with a 12-bit phase accumulator operating at a system frequency of 10MHz. Only the eight most-significant bits of the accumulator are used to index the LUT, which contains a full sine wave ...



- The system described above implements the following function: $x[n] = A \sin(2\pi f_v n T_S)$. What design changes or extensions are necessary to include a constant phase offset ϕ_v to realize the function: $x[n] = A \sin(2\pi f_v n T_S + \phi_v)$.
- Implement the system using SystemVerilog.
- Create a dedicated testbench to verify the correct functionality of the DDFS system implemented by simulating the design using Mentor Modelsim.
- Extend the system implemented in b) so that you can select between $\phi_v = \{0^\circ, 45^\circ, 90^\circ, 180^\circ\}$

Practical Implementation of DDFS Architectures

- **[Exercise]:** A wide range of RF techniques requires the parallel generation of quadrature signals composed of an 'in-phase' component $\sin(2\pi f_v nT_S)$ and an 'phase-quadrature' component $\cos(2\pi f_v nT_S)$. Quadrature signals, also denoted as I/Q signals are orthogonal to each other and can be therefore transmitted simultaneously and fully recovered.



- The aim of this exercise is to extend a basic DDFS implementation (quarter sine wave LUT, no phase offset), so that it incorporates the generation of both sine and cosine signals.

Practical Implementation of DDFS Architectures

- Keep in mind that FPGA internal memory supports dual-port operation as illustrated in the following code snippet. This enables the simultaneous access of two different LUT memory locations[†].

```
1.  logic [LUT_DATA_WIDTH-1:0] lut_memory [2**LUT_ADDR_WIDTH-1:0];
2.
3.  initial $readmemb("dual_port_lut_memory_init.txt", lut_memory);
4.
5.  always_ff@(posedge clk) begin
6.      q_a <= lut_memory[lut_addr_a];
7.      q_b <= lut_memory[lut_addr_b];
8.  end
```

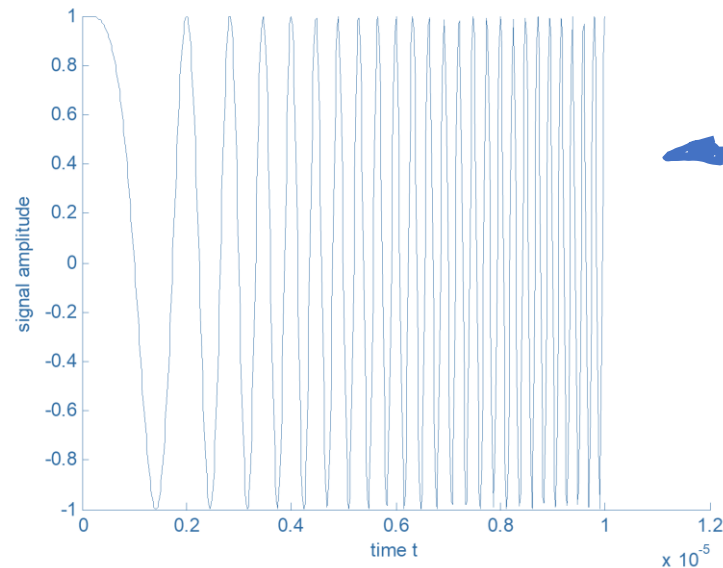
[†] and prevents the necessity of a secondary LUT memory ...

[SystemVerilog] Source Code Excerpt: dual_port_lut_memory.sv

- Sketch the RTL or block diagram of this particular DDFS structure.
- Implement the system using SystemVerilog (just the DDFS part which generates the cosine and sine wave signals).
- Create a dedicated testbench to verify the correct functionality of the DDFS system implemented by simulating the design using Mentor Modelsim.

Practical Implementation of DDFS Architectures

- **[Exercise]:** A chirp is a signal in which the frequency increases ('up-chirp') or decreases ('down-chirp') with time. The signal is termed 'chirp' because it sounds like the chirp of a bird when played through an audio speaker. In some literature, the term chirp is used interchangeably with sweep signal.



- The figure on the right hand side shows a linear sweep signal. Here the instantaneous frequency varies linearly with time:

$$\sin(2\pi f_v[n]nT_S) \text{ with } f_v[n] = f_0 + \beta n$$

where f_0 is the starting frequency (at time 0), and β is the rate of frequency change within the time interval T (also called chirp rate):

$$\beta = \Delta f / T$$

Practical Implementation of DDFS Architectures

- Consider the following scenario: We assume a starting frequency of 440Hz. The frequency should rise about 10Hz every 5ms. This rate remains constant and does not change over time.
 - a) Sketch the RTL or block diagram of this particular DDFS structure.
 - b) Implement the system using SystemVerilog (just the DDFS part which generates the cosine and sine wave signals).
 - c) Create a dedicated testbench to verify the correct functionality of the DDFS system implemented by simulating the design using Mentor Modelsim.

Introduction to DDFS Architectures

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

DDFS Architectures

Recommended Readings

Books, application notes, ...

- Sutherland, S., “RTL Modeling with SystemVerilog for Simulation and Synthesis: Using SystemVerilog for ASIC and FPGA Design”, CreateSpace Independent Publishing Platform, 2017
- Symons, P., “Digital Waveform Generation”, Cambridge University Press, 2013.
- Analog Devices, “All About Direct Digital Synthesis”, Analog Dialogue 38-08, 2004.